

用於物聯網環境之 開放式閘道器架構

國立成功大學工程科學系

鄧維光、楊皓宇

2025年5月29日



Outline

大綱>>

01/
容器化處理

02/
實驗評估

03/
案例規劃 - RFID應用

04/
參考資料與附錄

Outline

大綱>>

01/
容器化處理

02/
實驗評估

03/
案例規劃 - RFID應用

04/
參考資料與附錄

01 / 容器化預期效益

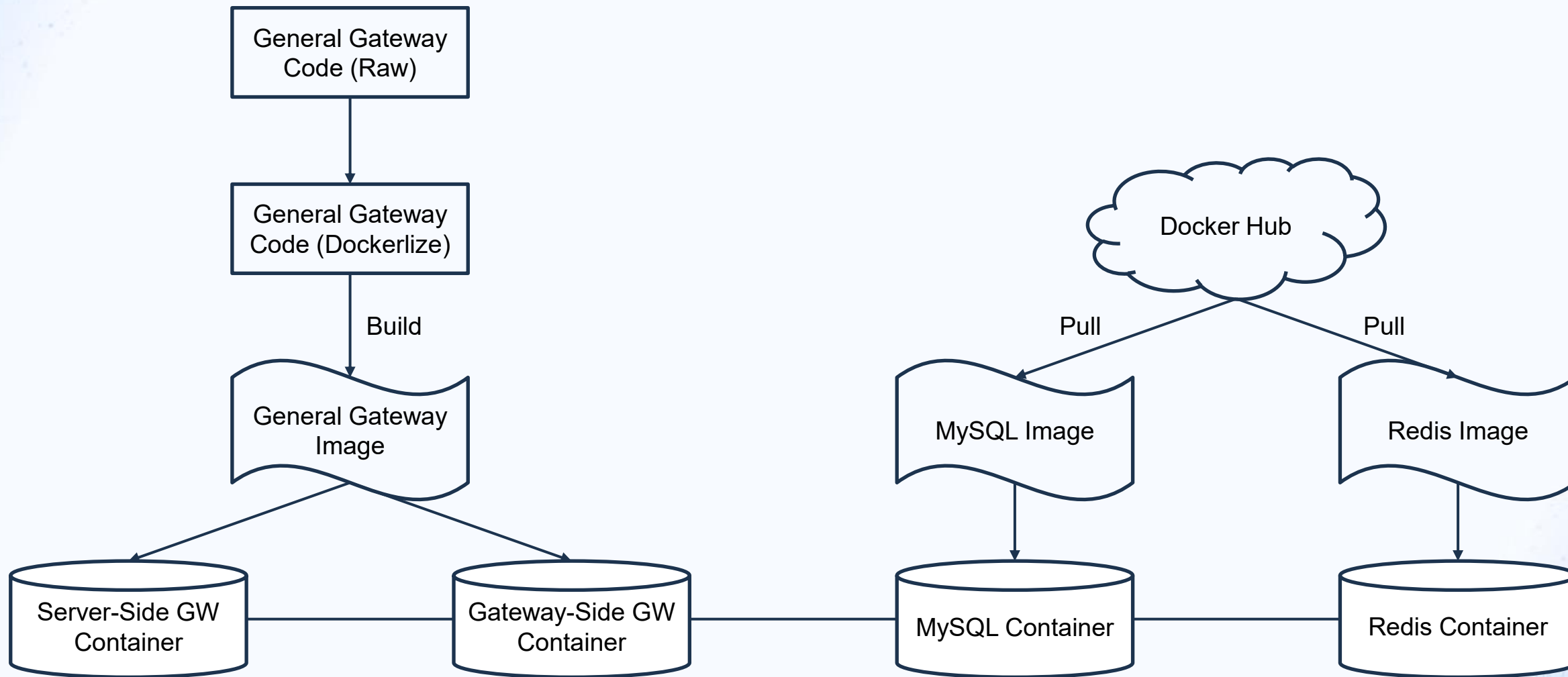
1. 穩健性 (Robustness) :

- 系統在異常情況下 (如設備故障、網路延遲、訊息錯誤) 仍能**維持正確與穩定運作**的能力
- 強調「容錯能力」、「高可用性」、「訊息驗證」

2. 可適應性 (Adaptability)

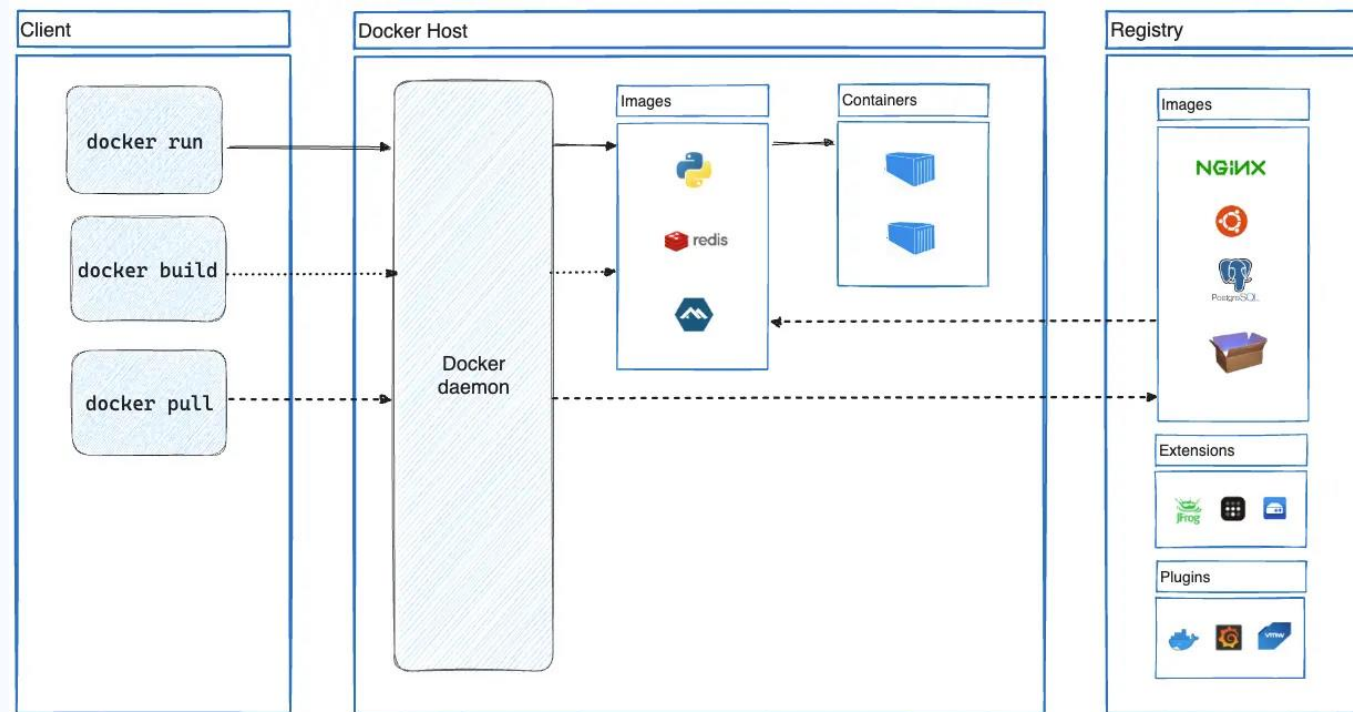
- 系統在不同環境下 (如不同硬體平台、部署場景、任務需求) **快速調整與擴充**的能力
- 強調「模組化」、「易部署性」、「可擴充性」、「快速部署」

01 / 容器化架構



01 / Docker運作流程

1. 使用者下指令 (build/run/pull)
2. Docker Daemon (dockerd)接收並執行
3. image來自本地或registry
4. container根據image啟動運行



Docker架構圖

Source: <https://docs.docker.com/get-started/docker-overview/>

01 / Dockerfile

- Dockerfile
 - 定義「**如何建構**」一個容器映像 (Image)
 - 內容包含：
 - 使用哪個基礎映像 (FROM)
 - 安裝哪些套件 (RUN)
 - 複製檔案 (COPY / ADD)
 - 指定啟動指令 (CMD / ENTRYPOINT)
 - 建構出的映像可以重複使用，確保開發與部署環境一致

```
1 FROM python:3.9-slim
2
3 WORKDIR /app
4
5 # 複製並安裝相依套件
6 COPY requirements.txt .
7 RUN apt-get update && \
8     apt-get install -y gcc default-libmysqlclient-dev && \
9     pip install --upgrade pip && \
10    pip install -r requirements.txt
11
12 # 複製專案程式碼
13 COPY . .
14
15 # 容器啟動指令
16 CMD ["python", "-u", "manage.py", "runserver", "-h", "0.0.0.0", "-p", "5000"]
```

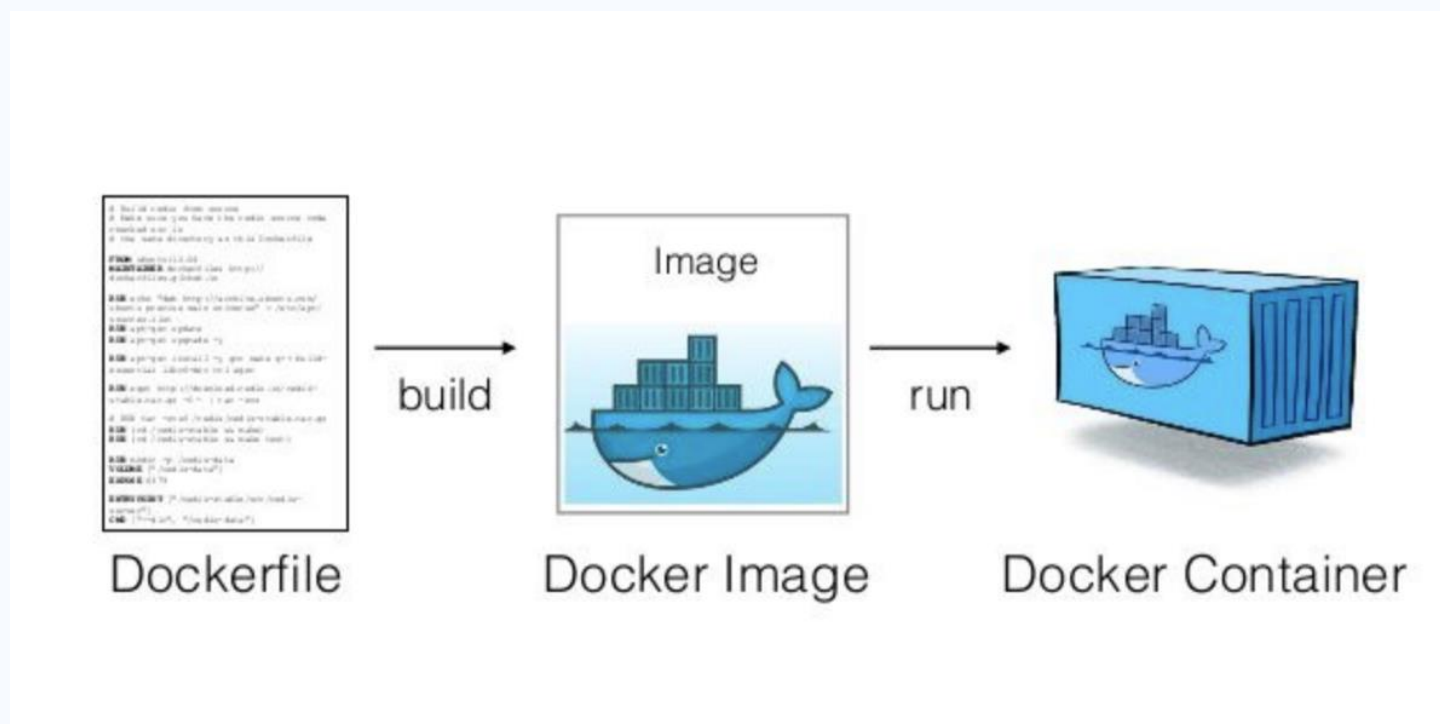
01 / docker-compose

- docker-compose
 - 定義「**如何啟動**」多個容器 (Container) 服務
 - 描述一組容器 (如 Web + 資料庫) 如何一起運作
 - 優點：
 - 一鍵啟動所有服務：docker-compose up
 - 管理多個容器間的網路與相依性
 - 適合開發、測試多服務系統

```
1  ∨ services:
    ▸ Run Service
2  ∨  server:
3      build: .
4      container_name: iot_server
5  ∨  ports:
6      - "5000:5000"
7  ∨  environment:
8      - CONFIG_FILE=/app/server_config.ini
9      - REDIS_HOST=iot_redis
10     - REDIS_PORT=6379
11 ∨  depends_on:
12     - mysql
13     - redis
14 ∨  volumes:
15     - ./data/mysql:/var/lib/mysql
16     - ./data/redis:/data
17
    ▸ Run Service
18 ∨ gateway:
19     build: .
20     container_name: iot_gateway
21 ∨  ports:
22     - "5001:5000"
23 ∨  environment:
24     - CONFIG_FILE=/app/gateway_config.ini
25     - REDIS_HOST=iot_redis
```


01 / Docker image

- 當透過Dockerfile建立一個Docker image時，實際上是將**應用程式所需的所有環境、套件、設定檔等**全部打包在一起，形成一個可攜、獨立的映像檔 (image)



Docker核心概念示意圖

Source: <https://cto.ai/blog/docker-image-vs-container-vs-dockerfile/>

Outline

大綱>>

01/
容器化處理

02/
實驗評估

03/
案例規劃 - RFID應用

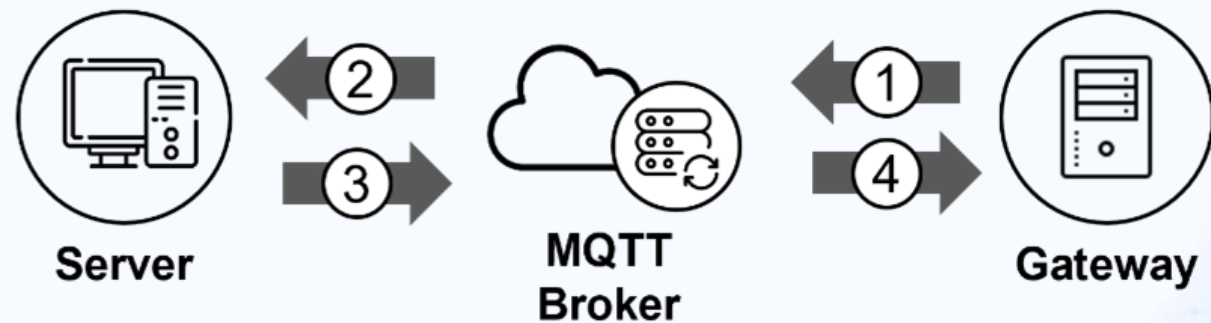
04/
參考資料與附錄

02 / 實驗評估

- 為驗證系統性能，規劃實驗聚焦於三大面向：即時性、穩定性、吞吐量
- 評估容器化前後及在不同環境條件下效能差異

02 / 即時性 - 通訊模組之傳輸速度

- 意義：評估訊息延遲，**判斷系統是否能滿足即時反應需求**，如工業設備控制、緊急事件通知
- 方法：單一Gateway連續發送10,000則訊息至Server
- 實驗一：測量路線1+2單向延遲時間
- 實驗二：測量路線1+2+3+4來回往返時間



02 / 穩定性 - 模擬封包遺失率比較通訊可靠性

- 意義：驗證系統在封包遺失情況下的錯誤處理與自我恢復能力，**確保訊息順利送達且不會等太久**，提升系統可信度
- 方法：單一Gateway連續發送10,000則訊息至Server，並以ACK機制模擬封包遺失，未收到ACK之Gateway會持續嘗試重新傳送訊息
- 實驗一：測量封包遺失率5%之重傳次數與花費時間
- 實驗二：測量封包遺失率10%之重傳次數與花費時間

- **模擬封包遺失率**：當接收到訊息時，生成一個介於 0~1 之間的隨機浮點數，若大於等於設定的封包遺失率，則該訊息視為有效訊息並進行後續處理，若小於，則視為無效訊息，並跳過該封包的處理

- 假設封包成功發送機率為 p ，失敗機率為 $1 - p$ ，則第 k 次發送成功機率為：

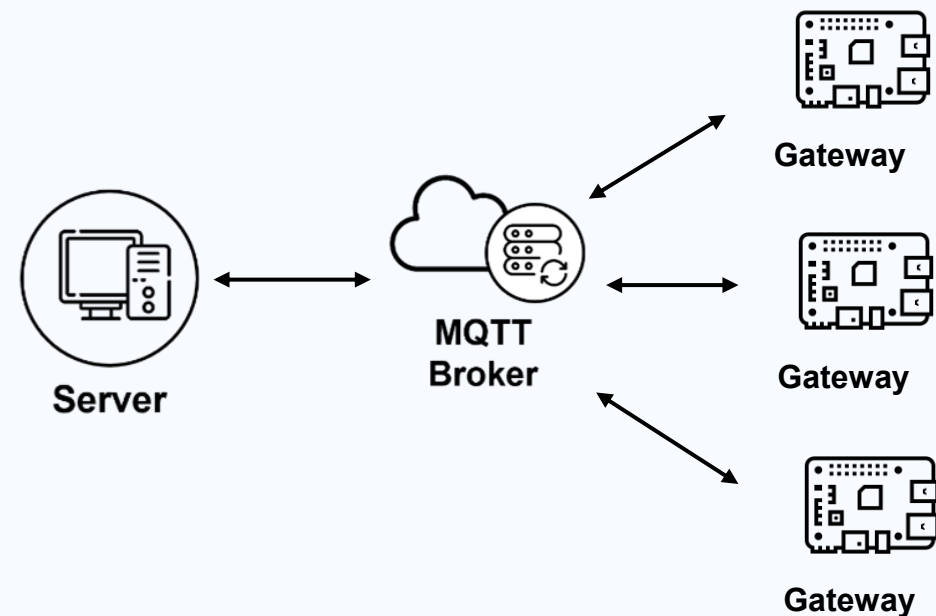
$$P(X = k) = (1 - p)^{k-1} \cdot p$$

因此**平均發送次數**為所有可能次數的加權平均：

$$E(X) = \sum_{k=1}^{\infty} k \cdot P(X = k) = \frac{1}{p}$$

02 / 吞吐量 - 量測伺服器訊息吞吐量

- 意義：測試系統在高負載下可處理的資料量，評估其在**大規模應用中的可擴展性與效能瓶頸**
- 方法：Server持續主動或被動發送訊息至多個Gateway
- 實驗一：測量Server主動向所有Gateway傳送訊息的單位時間訊息量
- 實驗二：利用ACK機制，測量Server接收並被動向所有Gateway傳送訊息的單位時間訊息量



Outline

大綱 > >

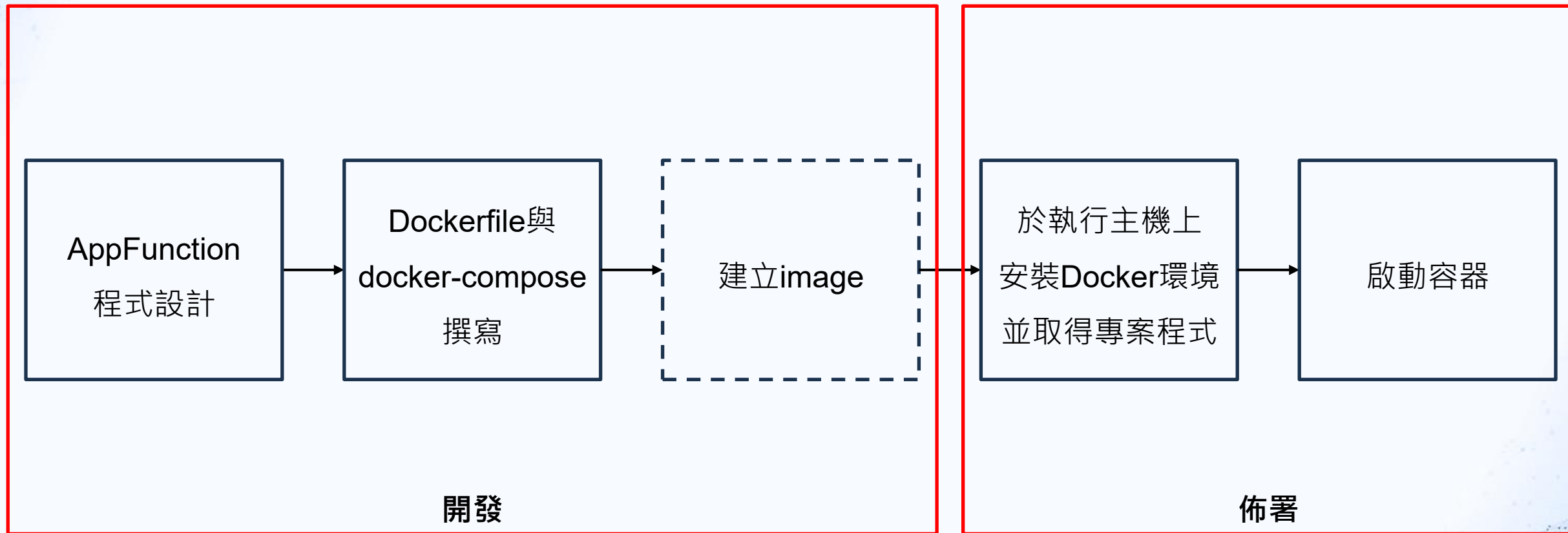
01/
容器化處理

02/
實驗評估

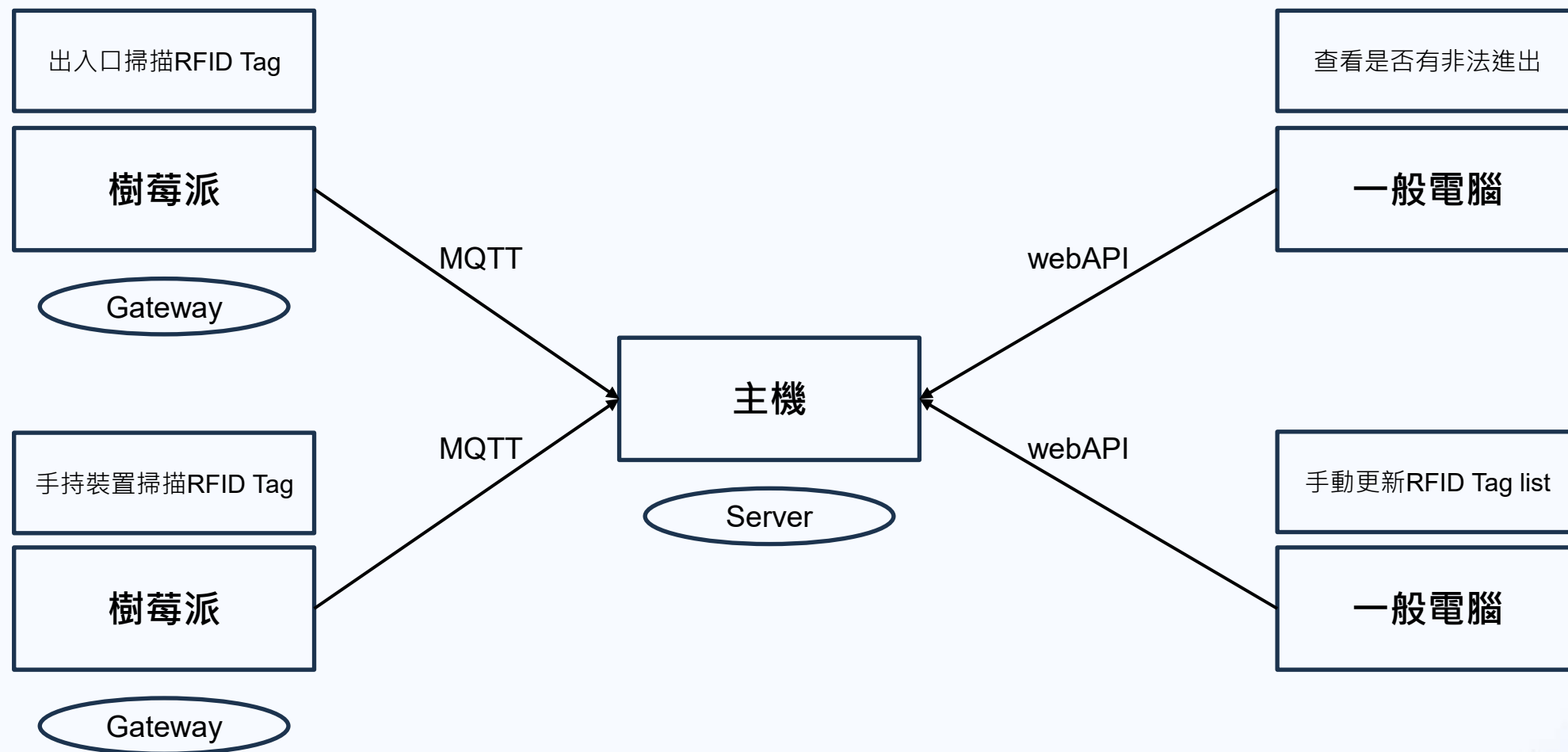
03/
案例規劃 - RFID應用

04/
參考資料與附錄

03 / 開發與佈署流程



03 / RFID應用架構



Outline

大綱>>

01/
容器化處理

02/
實驗評估

03/
案例規劃 - RFID應用

04/
參考資料與附錄



Outline

大綱>>

01/

專案現況概覽

02/

過往實驗

03/

未來規劃

04/

參考資料與附錄

01 / 專案目標

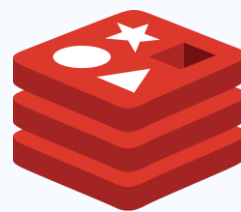
1. GW專案預期在現有程式架構下達到易部署與高穩定性的目標：

1. 提升系統部署便利性，整合Server與Gateway架構，確保於Linux與Windows環境皆可穩定執行，建立標準化部署流程與操作文件，降低未來維護與擴展成本
2. 系統穩定性測試與驗證，規劃與執行多項壓力與異常情境實驗，評估整體系統在各種情況及條件運行下的穩定性

01 / 新環境部署流程

1. 執行環境建置

- Python：安裝Python與各個相關套件
- MySQL：安裝MySQL並匯入資料表
- Redis：安裝Redis
- MQTT：建置MQTT Broker



redis



01 / 新環境部署流程

2. 修改設定檔中參數

- 修改setting.py與config.py的部分參數
 - Role/ROLE : Gateway或Server
 - server_id/SERVER_ID : Server的IP
 - GATEWAY_ID : Gateway的MAC address
 - GATEWAY_SN : Gateway的序號
 - BROKER_HOST : Broker的IP

```
setting.py > ...
1  BROKER_HOST = "140.116.39.171"
2  SERVER_ID = "140.116.39.171"
3  GATEWAY_ID = "11ea6335-d2bd-4678-9ca9-647b5574a11e"
4  ROLE = "Gateway"
5  GATEWAY_SN = 1
6
7  if ROLE == "Gateway":
8      DB_CONFIG = {
9          'host': 'localhost',
10         'user': 'root',
11         'password': 'root',
12         'db': 'fleet_gateway'
13     }
14 else:
15     DB_CONFIG = {
16         'host': 'localhost',
17         'user': 'root',
18         'password': 'root',
19         'db': 'fleet_server'
20     }
```

Gateway端setting.py參數設置

```
config.py > ...
1  import logging
2  from logging.handlers import RotatingFileHandler
3  import redis
4
5  R = redis.Redis('localhost')
6  role = "Gateway"
7  ROLE = "Gateway"
8
9  server_id = "140.116.39.171"
10 class BasicConfig(object):
11     # Flask (以及相關的擴展extension) 需要進行加密
12     SECRET_KEY = '421d7fa062a76ca25669e91923a3c79f'
13     # 設置是否在每次連接結束後自動提交數據庫中的變動。
14     # SQLAlchemy_COMMIT_ON_TEARDOWN = True
15     # 如果設置成 True (默認情況), Flask-SQLAlchemy 將會追蹤
16     SQLAlchemy_TRACK_MODIFICATIONS = True
17     # 可以用於顯式地禁用或者啟用查詢記錄。查詢記錄 在調試或者
18     SQLAlchemy_RECORD_QUERIES = True
19
20     @staticmethod
21     def init_app(app):
22         # 這程式主要的功能是希望每次運行程式後能將 log 續寫至
23         # 並在檔案達到 maxBytes 指定的大小後便新增新的log檔
```

Gateway端config.py參數設置

01 / 系統執行方式

1. 於Server與Gateway端輸入執行指令 (python manage.py runserver -h 0.0.0.0 -p 5000 -d) 後，Doorkeeper隨即開始不斷撈取新訊息

```
PS C:\Users\admin\Downloads\Fleet-Management-Test(send_msg v1.1)\Fleet-Management-Test(send_msg v1.1)> python manage.py runserver -h 0.0.0.0 -p 5000 -d
---Config---
ROLE: Gateway
SERVER: 140.116.39.171
DELIVER WAY: Ethernet
---Config---
Creating new instance: <general_gateway.main.GeneralGateway object at 0x000001E5628B3CD0>
general_gateway doorkeeper
doorkeeper: start
Connect MQTT Broker Successfully!!!

---on_connect---
---Connected with MQTT Broker---
Returned code: 0
Session is alive, present: 0
---mqtt_on_subscribe---
Gateway Subscribe: 140.116.39.171/to/11ea6335-d2bd-4678-9ca9-647b5574a11e/#
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://140.116.39.174:5000
Press CTRL+C to quit

_deal_task: has_task None
_deal_task: doing_task 0
Into _deal_task!!

TaskManager TASK_DICT_KEY: {}
Doorkeeper:fetch_need_publish_messages
TaskManager:fetch_need_deal_tasks
listener: 1
```

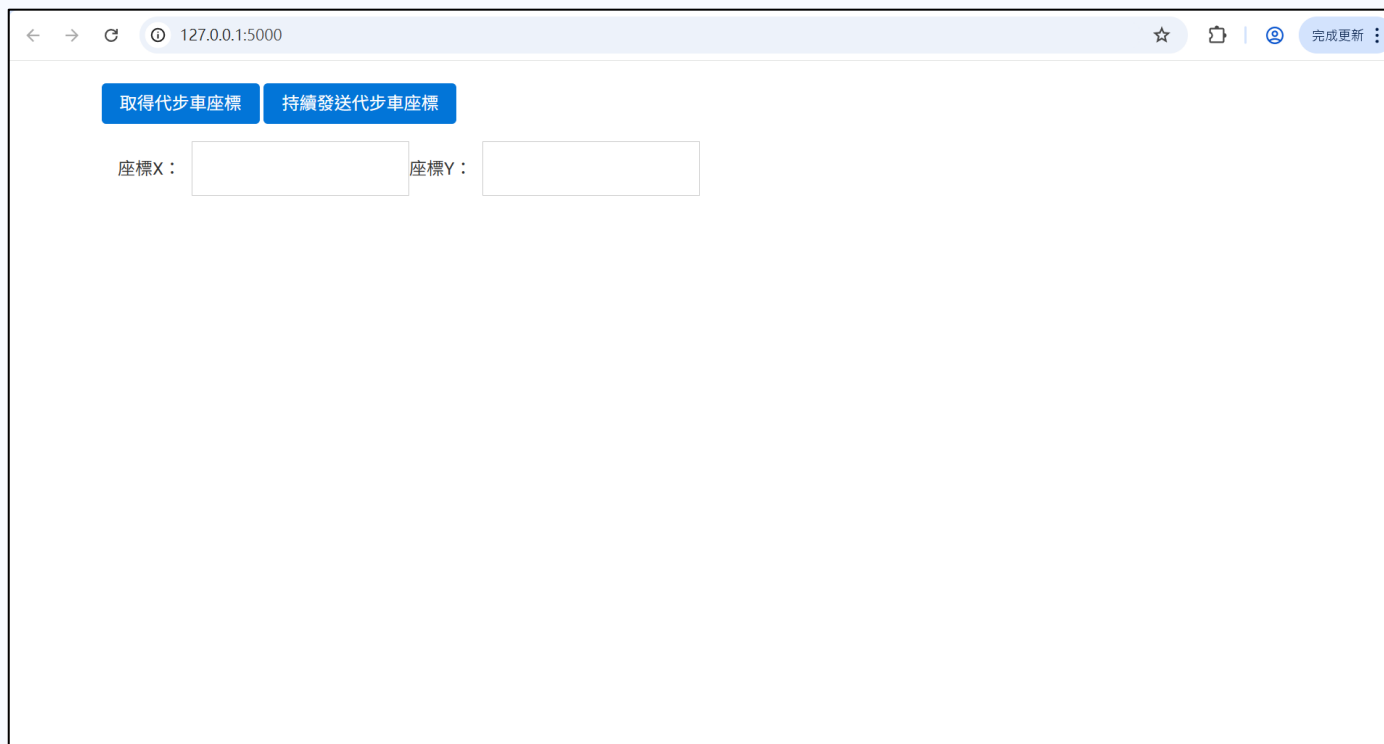
```
listener: 1
listener: Doorkeeper._send_message , _send_message , 2025-04-30 21:20:59.462782+08:00
_deal_task: has_task 0
_deal_task: doing_task 0

Doorkeeper:fetch_need_publish_messages
listener: 1
listener: Doorkeeper._send_message , _send_message , 2025-04-30 21:21:00.462782+08:00
_deal_task: has_task 0
_deal_task: doing_task 0
Doorkeeper:fetch_need_publish_messages
listener: 1
listener: Doorkeeper._send_message , _send_message , 2025-04-30 21:21:01.462782+08:00
_deal_task: has_task 0
_deal_task: doing_task 0
Doorkeeper:fetch_need_publish_messages
listener: 1
listener: Doorkeeper._send_message , _send_message , 2025-04-30 21:21:02.462782+08:00
_deal_task: has_task 0
_deal_task: doing_task 0
Doorkeeper:fetch_need_publish_messages
listener: 1
listener: Doorkeeper._send_message , _send_message , 2025-04-30 21:21:03.462782+08:00
_deal_task: has_task 0
_deal_task: doing_task 0
Doorkeeper:fetch_need_publish_messages
listener: 1
listener: Doorkeeper._send_message , _send_message , 2025-04-30 21:21:04.462782+08:00
```

程式執行畫面

01 / 系統執行方式

1. 透過App Function設計，以代步車應用為例，可透過Flask網頁發送與顯示接收到的訊息

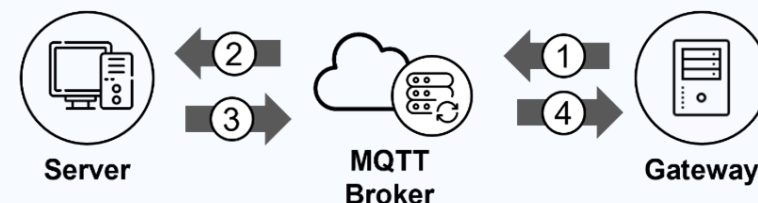


The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000". The page contains two blue buttons at the top: "取得代步車座標" (Get shared bicycle coordinates) and "持續發送代步車座標" (Continuously send shared bicycle coordinates). Below these buttons, there are two input fields labeled "座標X:" and "座標Y:". The "座標X:" field is currently empty, and the "座標Y:" field is also empty. The browser's address bar includes navigation icons (back, forward, refresh) and a "完成更新" (Update complete) button.

代步車測試介面

02 / 即時性 - 通訊模組之傳輸速度

1. 意義：評估訊息延遲，**判斷系統是否能滿足即時反應需求**，如工業設備控制、緊急事件通知
2. 結果：在網路連線穩定環境中，平均一則訊息的延遲 (Latency) 為0.072毫秒，提出的通訊架構網路延遲為毫秒等級。根據提出的 ACK 機制量測訊息往返時間，接收方需要對訊息進行解譯、更新資料表等等的操作，平均訊息往返時間 (Round-Trip Time) 為1秒
3. 實驗調整方向：測試不同Payload大小與頻率下的延遲變化；模擬網路擁塞情境，如限速或引入人為延遲



Method	訊息延遲	訊息往返
Max	0.310 ms	2110.210 ms
Median	0.073 ms	1033.417 ms
Min	0.014 ms	257.264 ms
Average	0.072 ms	1003.295 ms

02 / 穩定性 - 模擬封包遺失率比較通訊可靠性

1. 意義：驗證系統在封包遺失情況下的錯誤處理與自我恢復能力，**確保訊息順利送達且不會等太久**，提升系統可信度
2. 結果：因測試的裝置位於相同內網且網路連線穩定的環境，所以提出的通訊架構不會有訊息遺失的情況。透過連續發送 10,000 則訊息進行測試，並以 ACK 機制模擬封包遺失，結果顯示封包遺失率 5% 時，平均發送 1.05 次；封包遺失率 10% 時，平均發送 1.11 次即可成功傳輸訊息，即使在封包遺失率高的情況下，依然可透過訊息重傳機制保證所有訊息皆正確傳遞，且重傳後回應 ACK 時間不會因此延長
3. 實驗調整方向：觀察不同重送機制設定，如調整次數上限或間隔時間對整體成功率與延遲的影響；結合實際網路環境測試，如 Wi-Fi 不穩定；模擬長時間運行下的穩定性

- **模擬封包遺失率**：當接收到訊息時，生成一個介於 0~1 之間的隨機浮點數，若大於等於設定的封包遺失率，則該訊息視為有效訊息並進行後續處理，若小於，則視為無效訊息，並跳過該封包的處理

- 假設封包成功發送機率為 p ，失敗機率為 $1 - p$ ，則第 k 次發送成功機率為：

$$P(X = k) = (1 - p)^{k-1} \cdot p$$

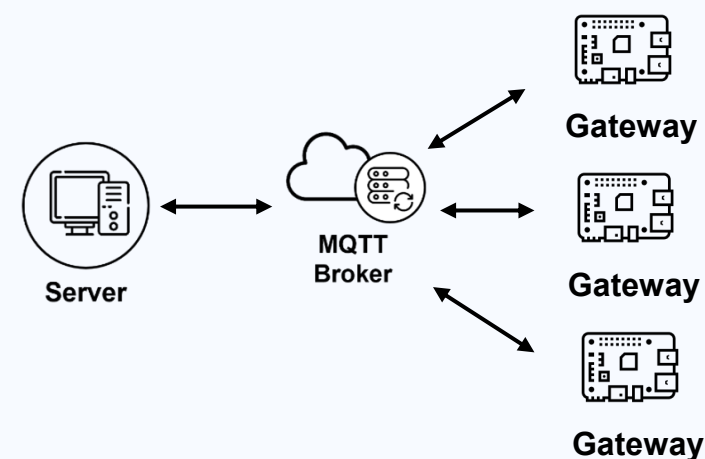
因此**平均發送次數**為所有可能次數的加權平均：

$$E(X) = \sum_{k=1}^{\infty} k \cdot P(X = k) = \frac{1}{p}$$

傳送次數 封包遺失率	1	2	3	4	5
5%	1.13 s	6.59 s	11.31 s	16.18 s	
10%	1.14 s	6.83 s	11.82 s	16.86 s	21.08 s

02 / 吞吐量 - 量測伺服器訊息吞吐量

1. 意義：測試系統在高負載下可處理的資料量，評估其在**大規模應用中的可擴展性與效能瓶頸**
2. 結果：以3台樹莓派模擬多裝置同時連線，並保持伺服器與閘道器間持續溝通，分別測試2種傳輸方向，利用 Ack 機制量測伺服器端單位時間訊息吞吐量。伺服器主動向閘道器發送訊息吞吐量約為30則/秒；伺服器被動接收並回應閘道器發送的訊息，吞吐量約為44則/秒。提出的通訊模組訊息收發受限於建立待發送訊息時頻繁的 INSERT 操作，導致系統吞吐量受到限制
3. 實驗調整方向：測試不同App Function負載下的影響，如是否會因其它線程任務造成效能下降；調整訊息發送頻率與Payload大小；增加Gateway數量進一步模擬大規模應用情境



Test	Method	Avg.	Max	Min
測試一	發送	29.98	59	23
	接收	29.98	49	14
測試二	發送	44.50	68	21
	接收	44.50	54	16

單位：則/秒

03 / 未來規劃

1. 目前若欲於新環境部署，須手動於**Server**和**Gateway**端進行執行環境的建置，過程可能出現預料外的錯誤且流程繁雜
2. 因此考慮引入**Docker**將環境與程式容器化，達到快速部署的目的
3. 容器化後測試環境可快速重建，便於模擬各種極端或異常狀況實驗 (如資源限制、網路中斷)，且可透過多容器部署模擬實際部署情境 (如數十個**Gateway**同時接入)

Docker化前後比較

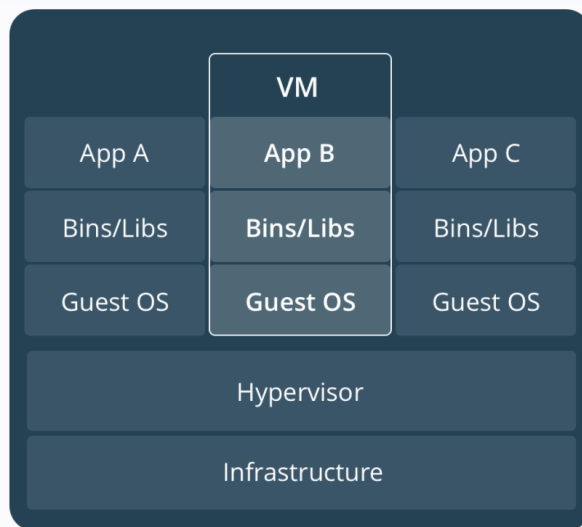
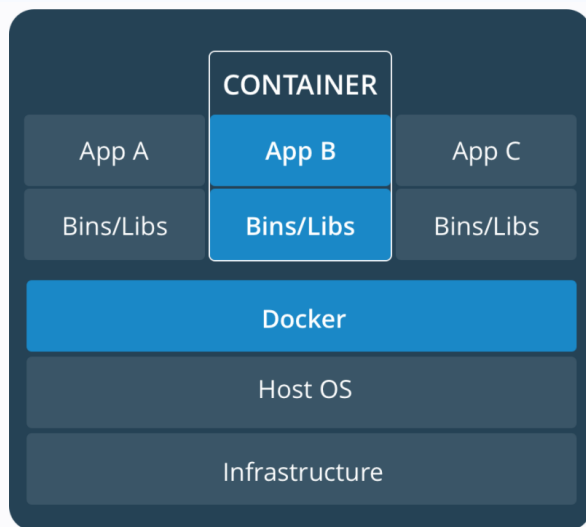
現在	容器化後
修改config.py和setting.py中常數	改成讀取環境變數，無須修改python檔
手動安裝套件並建立資料庫	Docker自動完成
指令手動執行程式 (python manage.py runserver -h 0.0.0.0 -p 5000 -d)	docker-compose up -d一鍵啟動
作業系統差異要調整環境	Windows/Linux均直接以Docker執行

03 / Docker簡介

1. Docker為一開源的容器平台，可解決環境不一致問題，且可跨平台部署並快速啟動
2. Docker大致由四個部分所組成：Image (映像檔)、Container (容器)、Dockerfile (建置腳本)、Docker Hub (雲端倉庫)

03 / Docker vs. VM

1. 虛擬機 (VM) 需要完整的OS組成：Kernel (核心)、User Space (用戶空間)、開機程序 (Bootloader + Init 系統)
2. Docker Image僅包含User Space，其僅依賴主機 (host) 的kernel，所以無需攜帶自己的kernel 和bootloader



項目	傳統虛擬機 (VM)	Docker容器
啟動速度	慢	快
系統資源使用量	高	低
隔離性	高	高
可攜性	低	高

Docker與VM比較

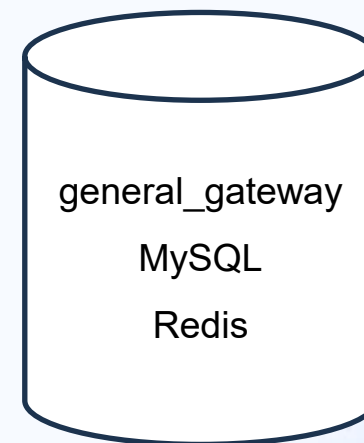
Source: <https://www.docker.com/resources/what-container/>

03 / Docker容器規劃

1. 合併容器缺點：

- 可維護性差：更新資料庫時會影響整個容器
- 失去容器隔離好處：資料庫出錯可能連帶導致主程式崩潰
- 容器變胖：一個容器塞三個服務，**image**更難管理
- 擴充困難：無法只水平擴展某個部分 (如多個Redis)

2. 僅在設備資源限制時考慮合併容器，如CPU、RAM、儲存空間非常有限的情況





結案簡報

Outline

大綱>>

01/

物聯網閘道器概覽

02/

開放式閘道器介紹

03/

各項應用之系統架構圖

04/

開放式閘道器優化規劃

05/

參考資料與附錄

Outline

大綱>>

01/

物聯網閘道器概覽

02/

開放式閘道器介紹

03/

各項應用之系統架構圖

04/

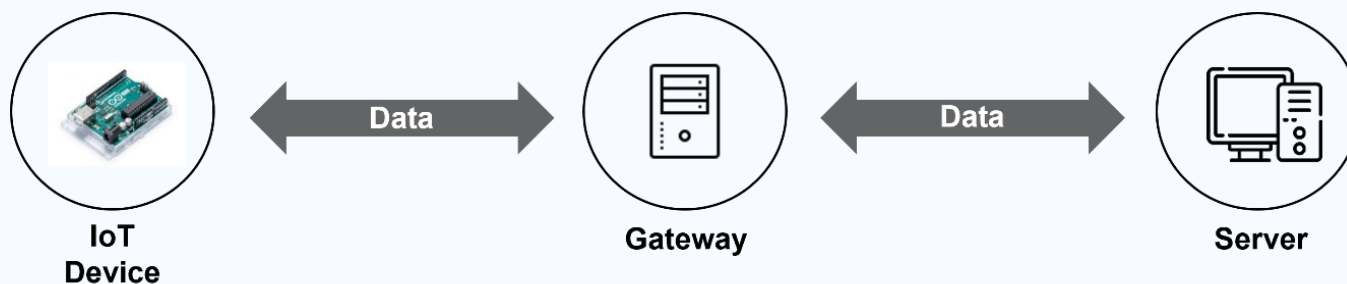
開放式閘道器優化規劃

05/

參考資料與附錄

研究背景

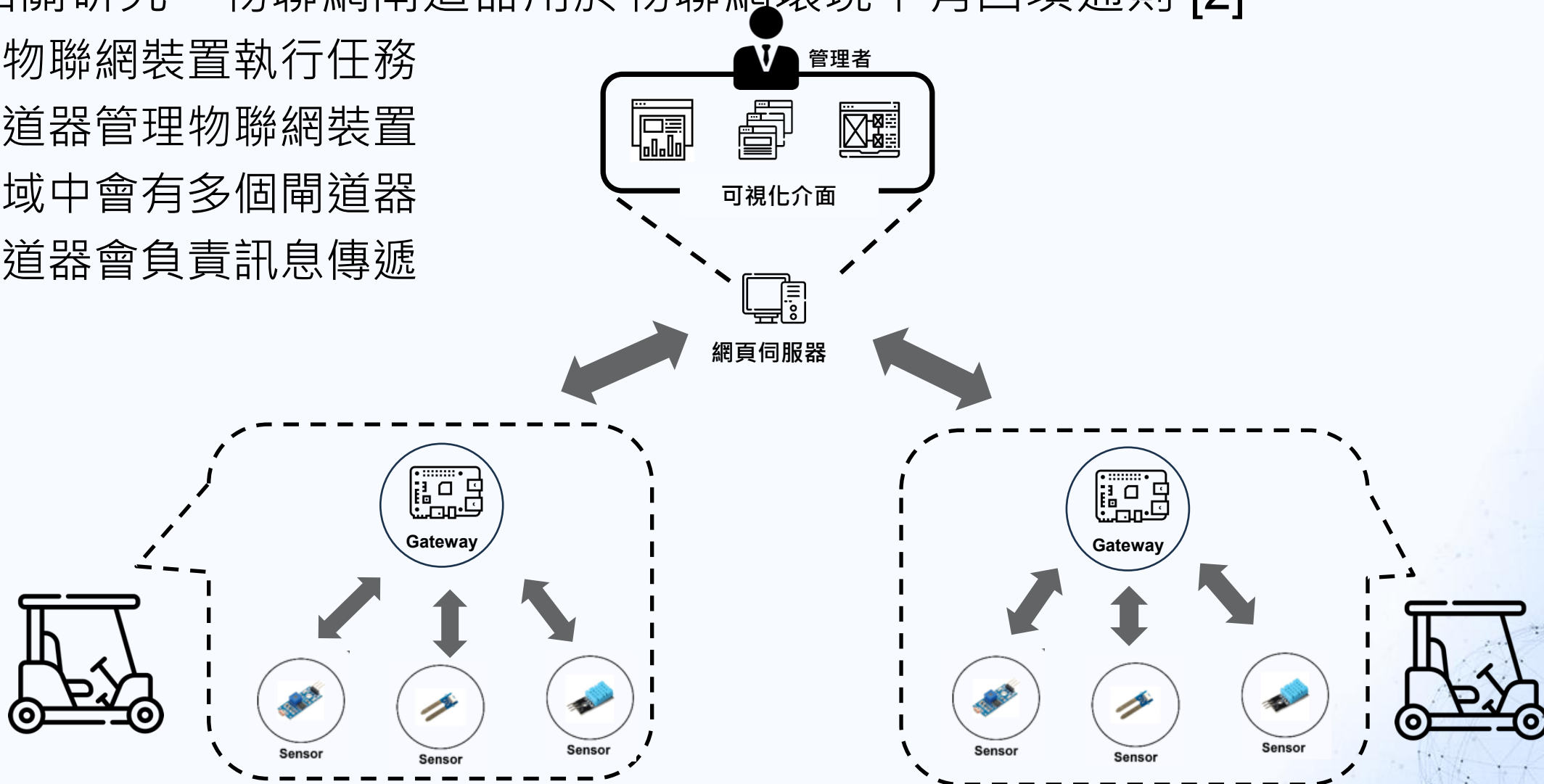
- 面對物聯網 (Internet of Things, IoT) 的興起，物聯網規模逐漸擴大，伺服器必須與更多的物聯網裝置進行訊息傳輸，造成伺服器的負擔增加
- 物聯網閘道器 (IoT Gateway) 的出現，則可協助伺服器控管物聯網裝置
- 物聯網閘道器是一台微型電腦，能夠儲存裝置的數據、控管裝置狀態、協助伺服器與裝置之間的資料傳輸 [1]



01 / 物聯網環境的四項通則

- 根據相關研究，物聯網閘道器用於物聯網環境中有四項通則 [2]

1. 由物聯網裝置執行任務
2. 閘道器管理物聯網裝置
3. 場域中會有多個閘道器
4. 閘道器會負責訊息傳遞



附加於物聯網閘道器通訊架構的三項特點

- 根據通則設計物聯網閘道器的軟體架構，並附加此架構三項特點

特點	說明	舉例
通用性 (Generality)	各物聯網環境皆通用的閘道器通訊架構	已應用於倉儲管理、電力管理、車隊管理
可靠性 (Reliability)	訊息於網路連線穩定環境下延遲為 毫秒等級 ，不穩定的環境中也可透過訊息重傳機制，在恢復連線後完成訊息傳輸	在網路不穩定的工廠環境中，伺服器接收閘道器的電力抄表資訊
延展性 (Scalability)	伺服器不需停機即可增減閘道器，每秒可回應 44 則訊息	在定義的規格下，可任意增減或更新場域內車輛與其車輛資訊，而伺服器不需停機，仍然可以控管車輛

Outline

大綱>>

01/

物聯網閘道器概覽

02/

開放式閘道器介紹

03/

各項應用之系統架構圖

04/

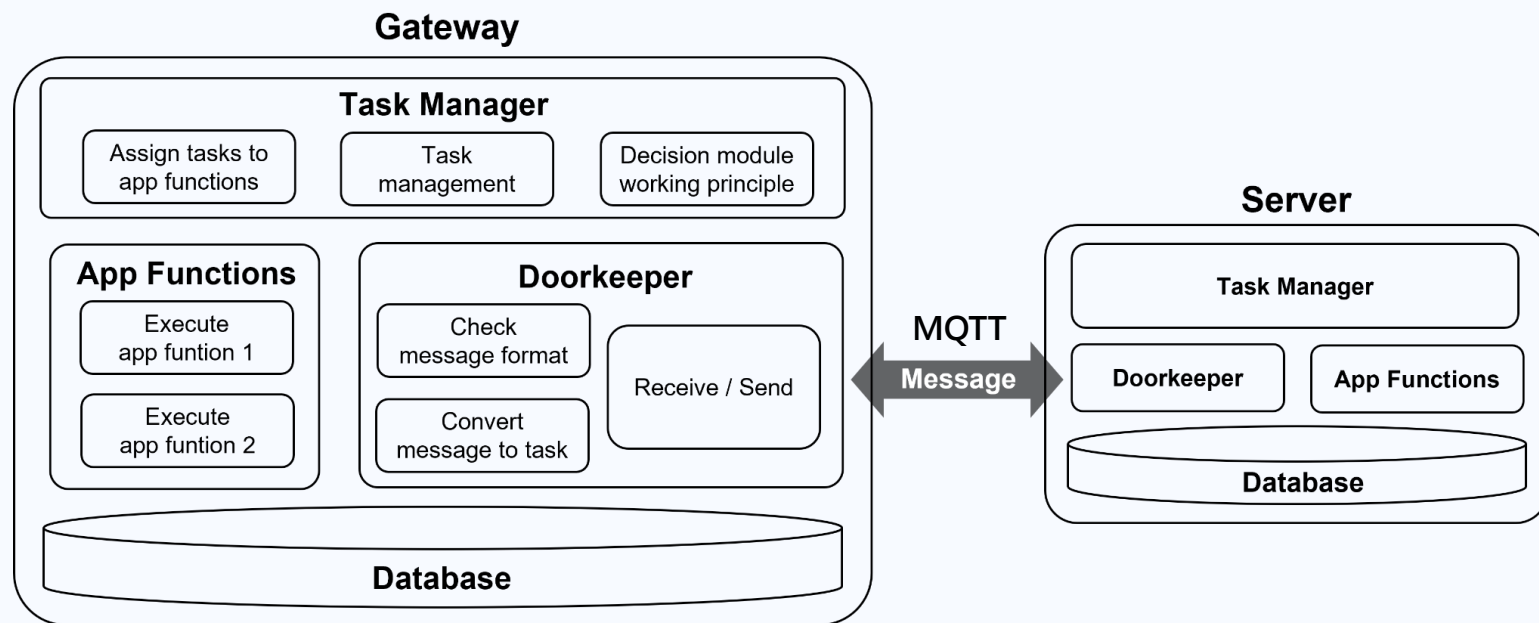
開放式閘道器優化規劃

05/

參考資料與附錄

各物聯網環境皆通用的閘道器通訊架構

- 依照通則建立物聯網閘道器通訊架構，伺服器與閘道器採用**相同的架構**進行訊息傳輸與裝置控管
- 以模組化進行架構設計，更換場景時只需少量修改程式碼，即可導入物聯網場域中
- 目前已經應用在三個場景之中
 - 智慧倉儲：RFID Reader進行貨品揀取與盤點貨架上貨品
 - 電力管理：控制電力閘開關與收集用電資訊等等
 - 車隊管理：追蹤代步車即時定位與代步車相關資訊等等

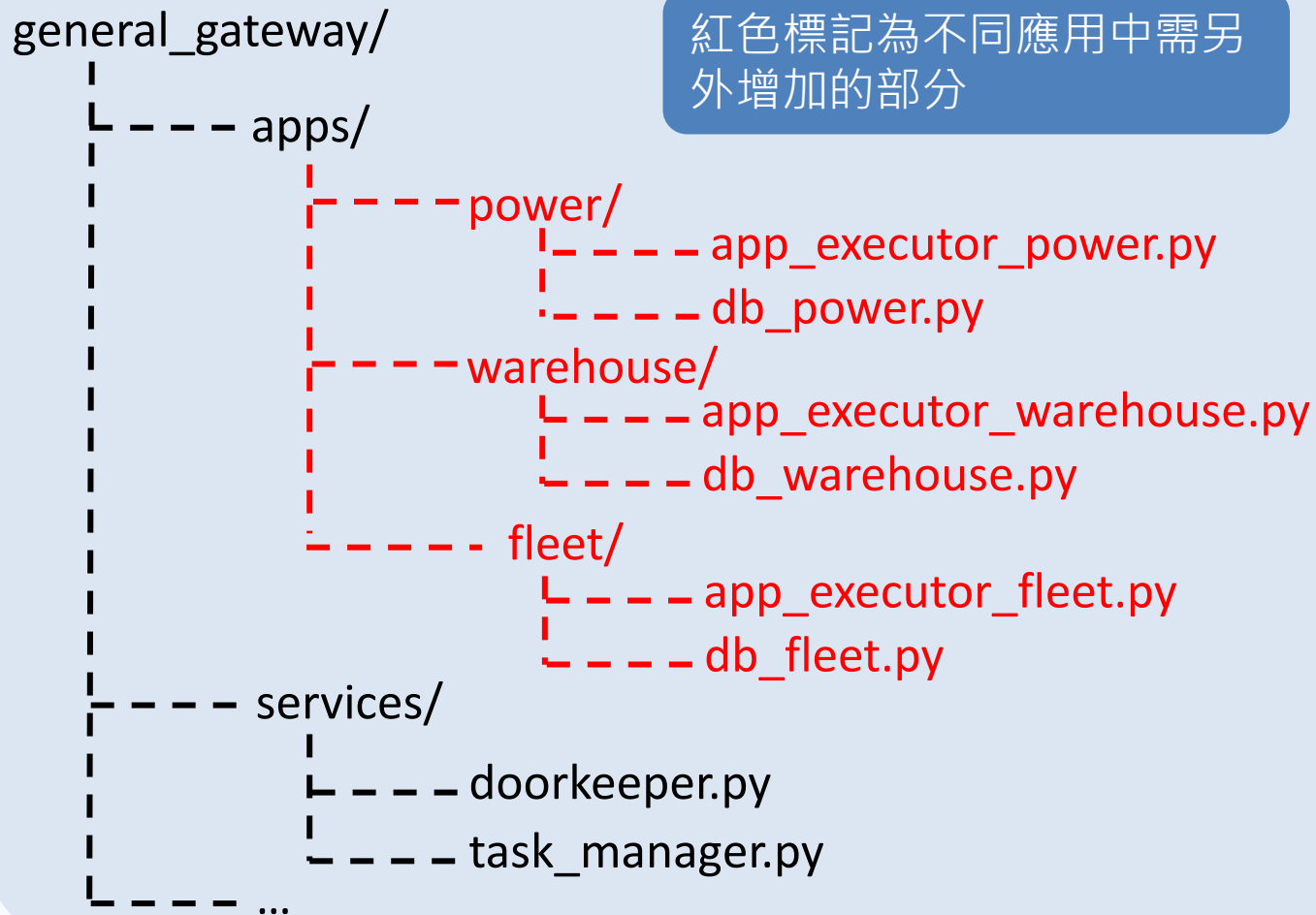


- **Doorkeeper**：負責收發訊息，檢查訊息格式，提取訊息中攜帶的任務
- **Task Manager**：負責分派任務給對應的App Functions並管理其進度
- **App Functions**：執行任務，例如回傳定位座標

快速導入物聯網場域

開放式閘道器模組架構

紅色標記為不同應用中需另外增加的部分

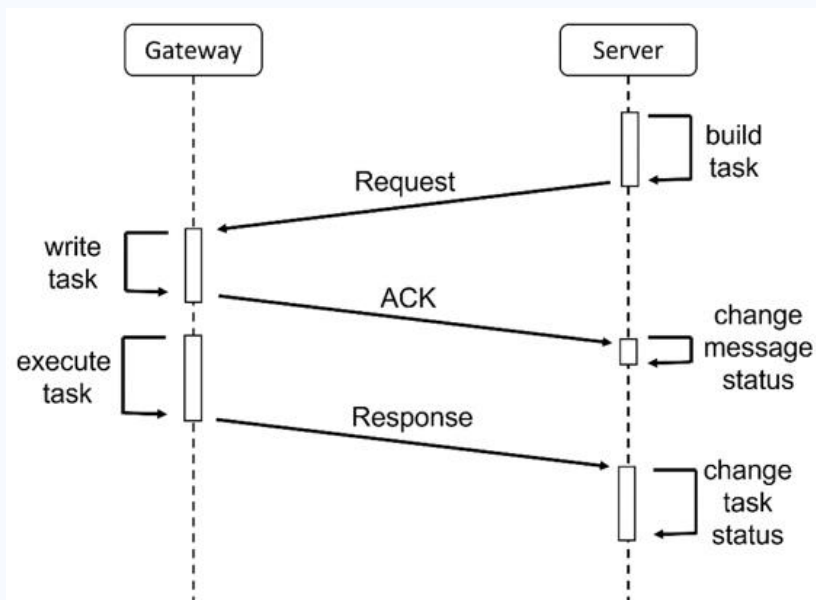


- Doorkeeper與Task Manager負責訊息傳遞與任務管理，程式碼能通用於各個場域，因此導入場域時只需在App Functions中開發特定的業務邏輯 (附錄八 ~ 十二)
- apps/：存放特定場域的App Functions
 - power：電力管理的App functions和相關資料表操作
 - warehouse：倉儲管理的App functions和相關資料表操作
 - fleet：車隊管理的App functions和相關資料表操作
- services/：模組核心
 - doorkeeper：收發訊息與訊息格式檢查
 - task_manager：管理任務並派發給對應的App functions

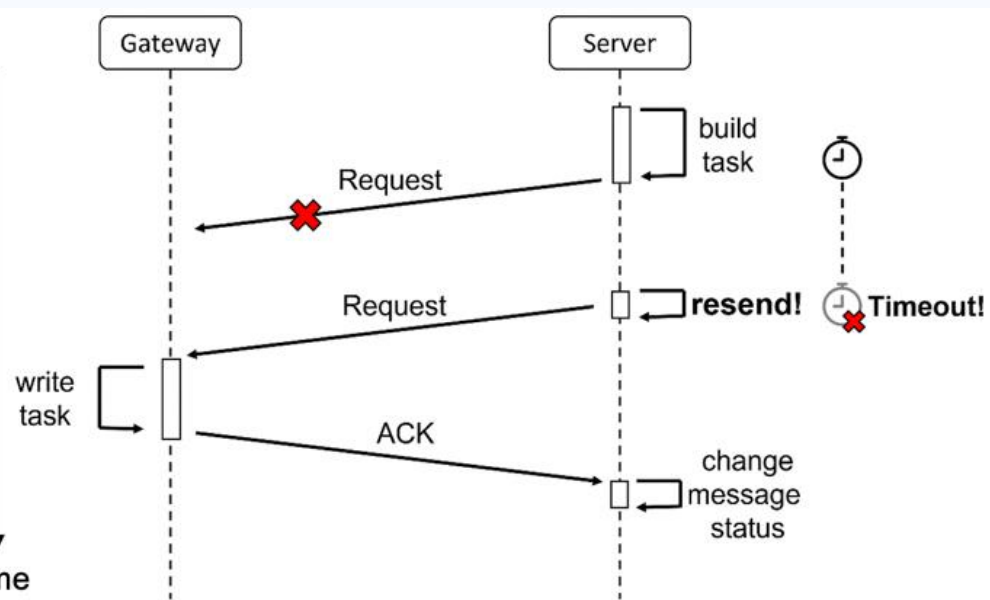
可自動重傳訊息確保訊息正確傳遞

- 需要發送訊息會保存在閘道器中，而我們設計兩種機制以確保訊息傳遞時的可靠性
 - Acknowledgement (ACK)：在收件方收到訊息時，會馬上回覆ACK給寄件方
 - Timeout: 寄件方需要在規定的時間 (5秒) 內收到ACK，若超過規定時間則會重傳訊息
- 若連續未收到ACK達 6 次，則會停止嘗試重傳，待10分鐘後再試一次

成功傳輸流程

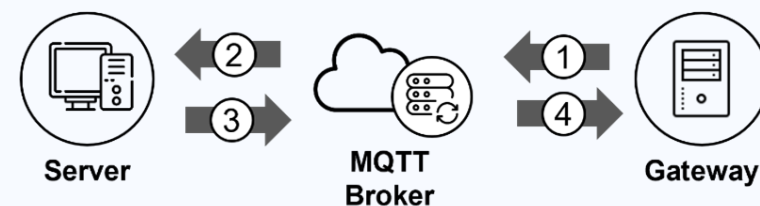
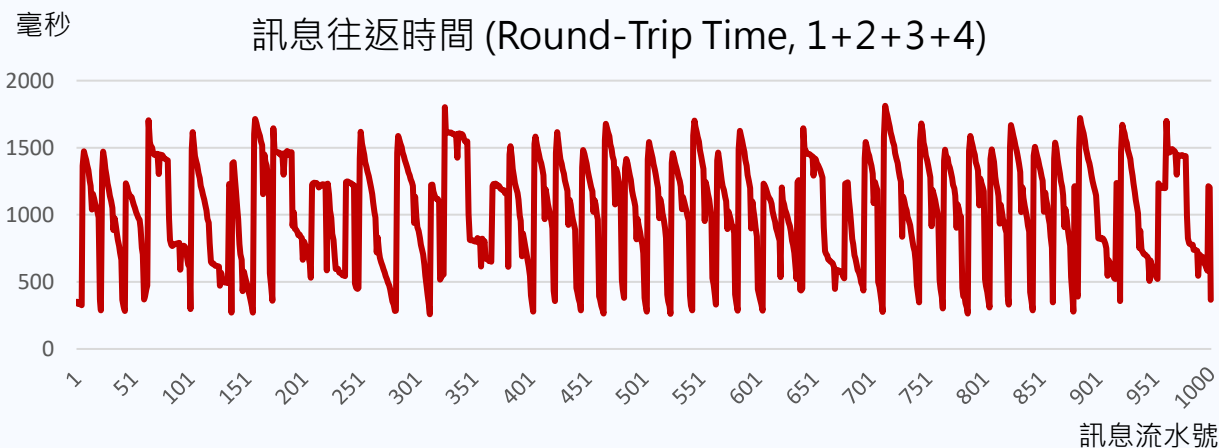
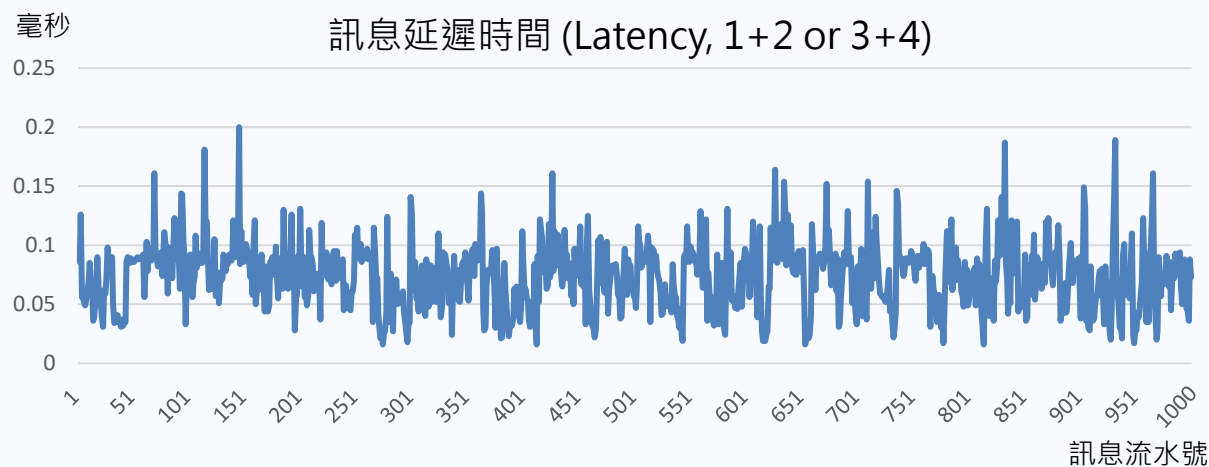


訊息重傳流程



通訊模組之傳輸速度

- 在網路連線穩定環境中，平均一則訊息的**延遲 (Latency)** 為 0.072 毫秒，提出的通訊架構網路延遲為**毫秒等級**
- 根據提出的 **ACK** 機制量測訊息往返時間，接收方需要對訊息進行解譯、更新資料表等等的操作，因此平均**訊息往返時間 (Round-Trip Time)** 為 1 秒，而訊息延遲性對於使用者**即時性高**



Method	訊息延遲	訊息往返
Max	0.310 ms	2110.210 ms
Median	0.073 ms	1033.417 ms
Min	0.014 ms	257.264 ms
Average	0.072 ms	1003.295 ms

模擬封包遺失率比較通訊可靠性

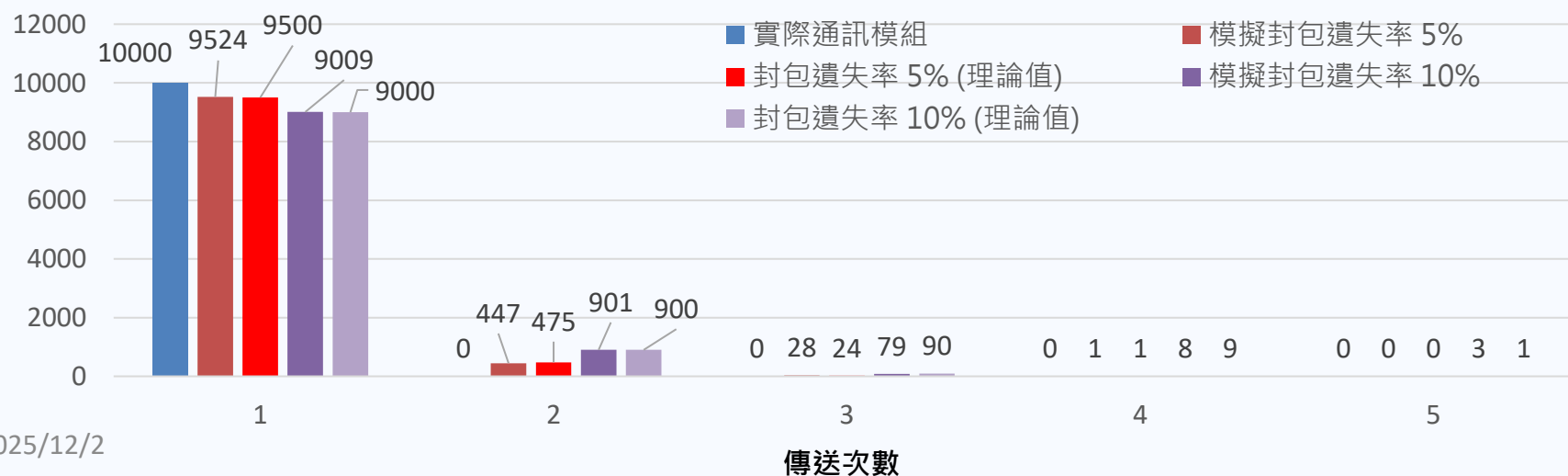
- 測試的裝置位於相同內網且網路連線穩定的環境，提出的通訊架構不會有訊息遺失的情況
- 連續發送 10,000 則訊息進行測試，並以 ACK 機制模擬封包遺失，結果顯示：

- 封包遺失率 5%，平均發送 1.05 次
- 封包遺失率 10%，平均發送 1.11 次

即可成功傳輸訊息，逾時 (Timeout) 設定為 5 秒內需收到回應的 ACK，即使在封包遺失率高的情況，可透過訊息重傳機制**保證所有訊息皆正確傳遞**，且重傳後回應 ACK 時間不會因此延長

傳送次數 封包遺失率	1	2	3	4	5
5%	1.13 s	6.59 s	11.31 s	16.18 s	
10%	1.14 s	6.83 s	11.82 s	16.86 s	21.08 s

訊息傳送次數分佈圖



- 模擬封包遺失率：**當接收到訊息時，生成一個介於 0~1 之間的隨機浮點數，若大於等於設定的封包遺失率，則該訊息視為有效訊息並進行後續處理，若小於，則視為無效訊息，並跳過該封包的處理

- 假設封包成功發送機率為 p ，失敗機率為 $1 - p$ ，則第 k 次發送成功機率為：

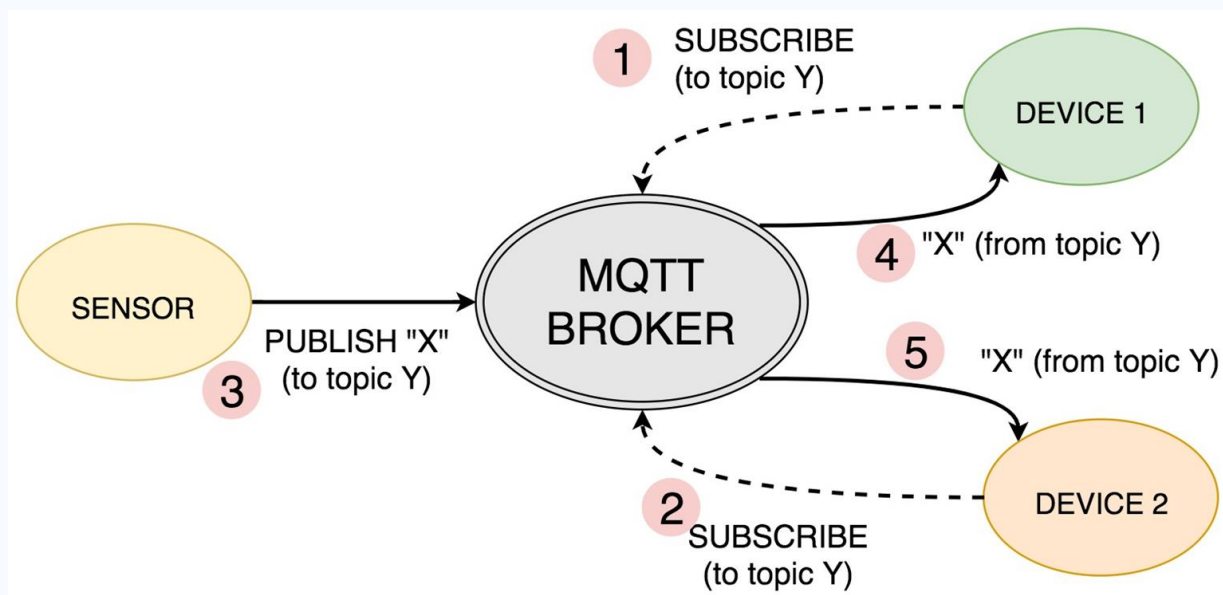
$$P(X = k) = (1 - p)^{k-1} \cdot p$$

因此平均發送次數為所有可能次數的加權平均：

$$E(X) = \sum_{k=1}^{\infty} k \cdot P(X = k) = \frac{1}{p}$$

MQTT通訊協定

- MQTT (Message Queuing Telemetry Transport) 是一個輕量級的發布/訂閱 (publish/subscribe) 消息通訊協議，在物聯網中十分常見
 - 發布/訂閱模型: 不同於傳統的客戶端-伺服器模型，MQTT 使用發布/訂閱模型，在此模型中，設備可以是消息的發布者、訂閱者或兩者都是
 - **Broker:** 中心節點，負責接收所有的消息和將它們分發到相應的訂閱者，常見的MQTT broker包括Mosquitto、HiveMQ 等
 - **Topics:** 訊息的標識，允許有訂閱特定Topic的發布者和訂閱者能夠收到訊息



伺服器不需停機即可增減閘道器數量

- MQTT為應用層的通訊協定，因此在分層架構中，底層的網路方式都可支援，不會影響到訊息的傳輸
- 訊息都會由MQTT Broker進行轉發，因此伺服器與閘道器互相訂閱Topic即可達到雙向溝通的目的
- 重新定義MQTT的Topic格式為三個層級 (伺服器/方向/閘道器)， “#” 代表伺服器訂閱所有閘道器的主題
- 一台伺服器可管理多個閘道器，伺服器只需訂閱一次Topic，便可在**不停機的狀況下增減閘道器數量**，也能**確保訊息傳遞**，藉此達到延展性

採用MQTT
進行訊息傳輸

OSI Model



Device(s)



Gateway(s)

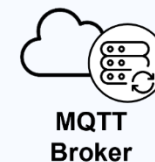


Subscribe
X/to/A1

Subscribe
X/to/A2

Subscribe
Y/to/B1

Subscribe
Y/to/B2



MQTT
Broker

重新定義Topic為三個層級

- Server傳到Gateway：ServerID/to/GWID
- Gateway傳到Server：ServerID/from/GWID

Server(s)



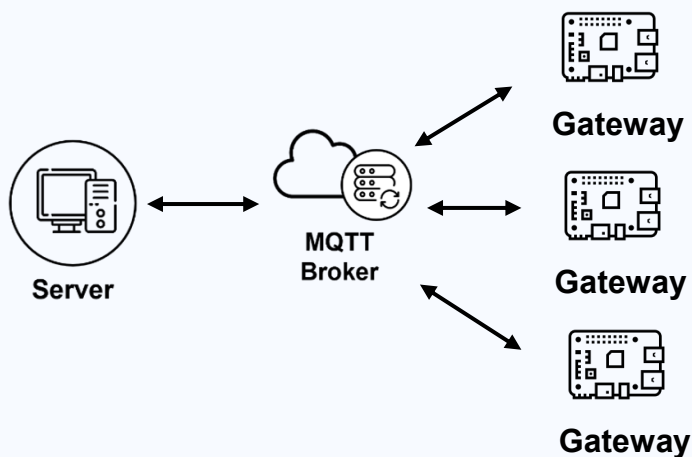
Subscribe
X/from/#

Subscribe
Y/from/#

- Subscribe only once
- X manages A1 and A2
- Y manages B1 and B2

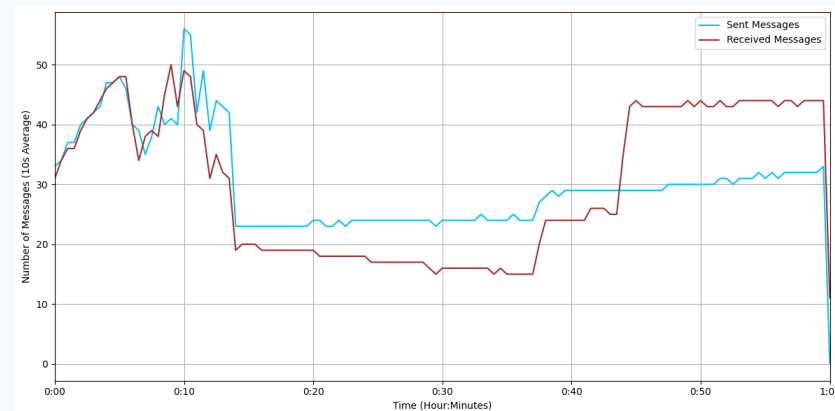
量測伺服器訊息吞吐量

- 使用 3 台樹莓派模擬多裝置同時連線，並保持伺服器與閘道器間持續溝通，以此定義提出的通訊架構之裝置管控能力
- 分別測試 2 種傳輸方向，利用 Ack 機制量測伺服器端單位時間訊息吞吐量：
 - 測試一：伺服器主動向閘道器發送訊息，吞吐量約為 30 則/秒
 - 測試二：伺服器被動接收並回應閘道器發送的訊息，吞吐量約為 44 則/秒
- 提出的通訊模組訊息收發受限於建立待發送訊息時頻繁的 INSERT 操作，導致系統吞吐量受到限制

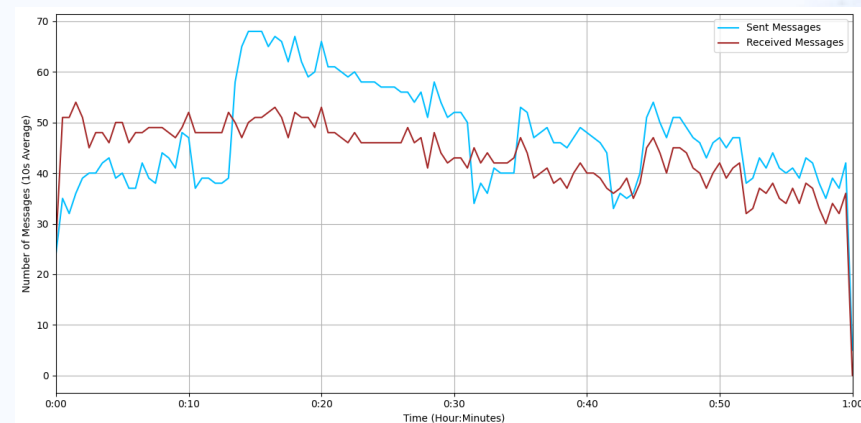


Test	Method	Avg.	Max	Min
測試一	發送	29.98	59	23
	接收	29.98	49	14
測試二	發送	44.50	68	21
	接收	44.50	54	16

單位：則/秒



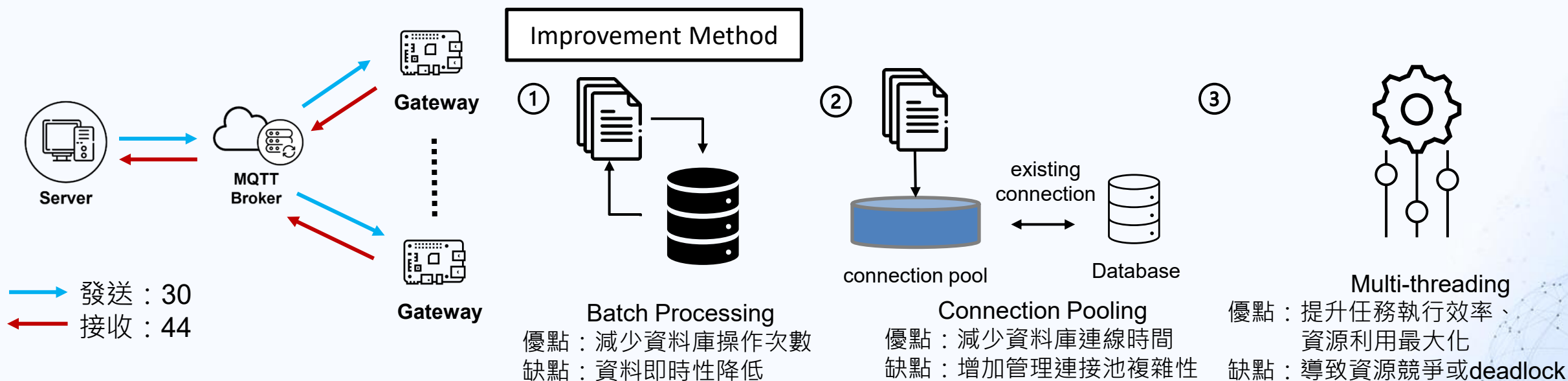
測試一：伺服器端訊息吞吐量



測試二：伺服器端訊息吞吐量

伺服器可監控裝置數量

- 伺服器每秒可主動發送30則訊息，接收後被動回傳44則
- 訊息吞吐量瓶頸在於每次收發訊息後需進行資料庫操作，高頻率操作資料庫導致讀寫效率下降
- 若要進一步提升系統吞吐量，可以考慮以下改善方法：
 1. 訊息批次處理：集中處理多筆訊息以減少資料庫操作次數，從而提高吞吐量
 2. 減少資料庫連線次數：採用資料庫連接池(connection pool) 減少連接資料庫的開銷
 3. 提升系統併發能力：增加執行緒數量，提升訊息處理的效率



Outline

大綱>>

01/

物聯網閘道器概覽

02/

開放式閘道器介紹

03/

各項應用之系統架構圖

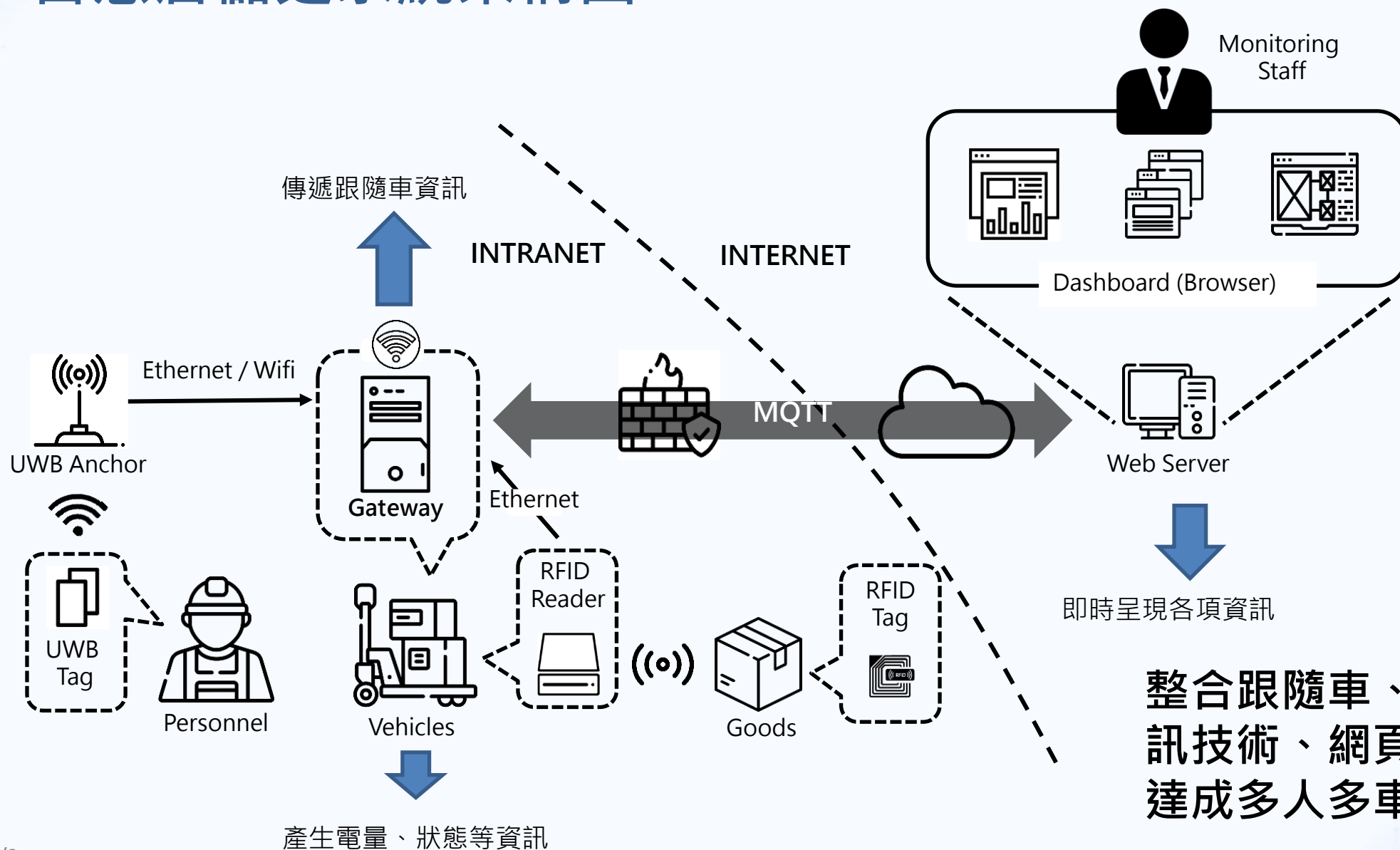
04/

開放式閘道器優化規劃

05/

參考資料與附錄

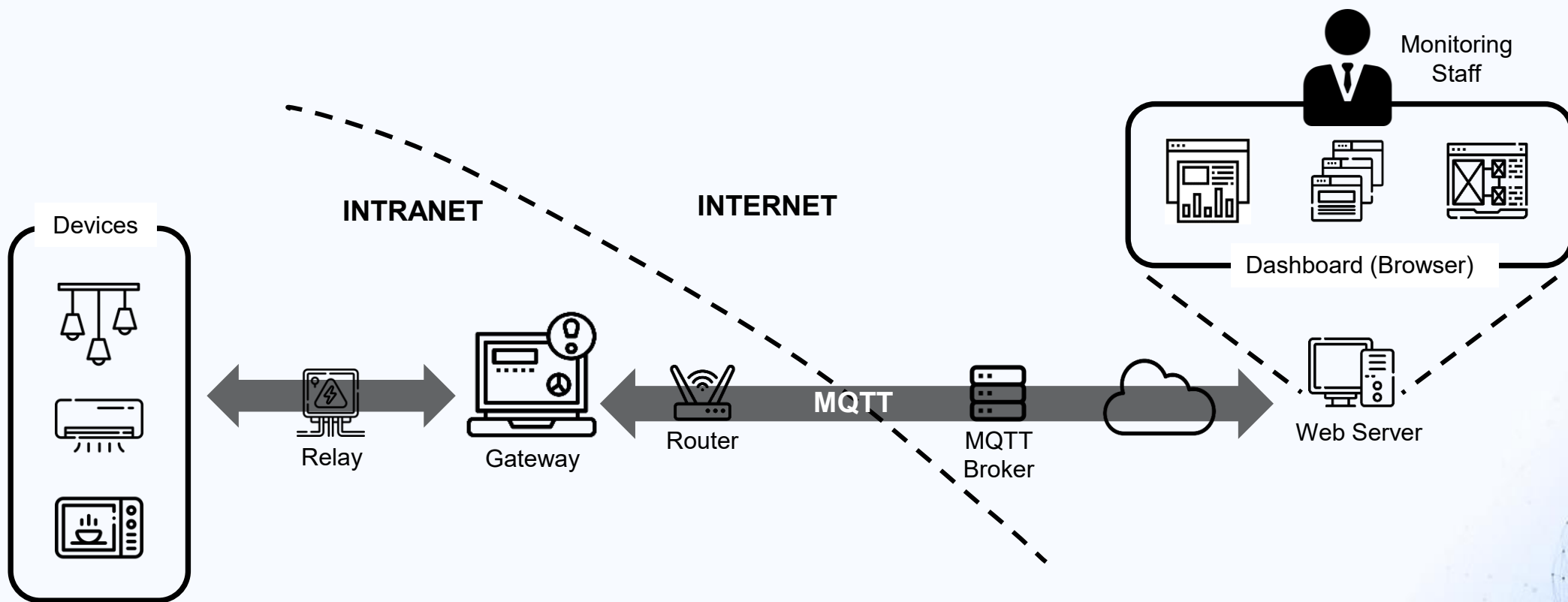
03/ 智慧倉儲之系統架構圖



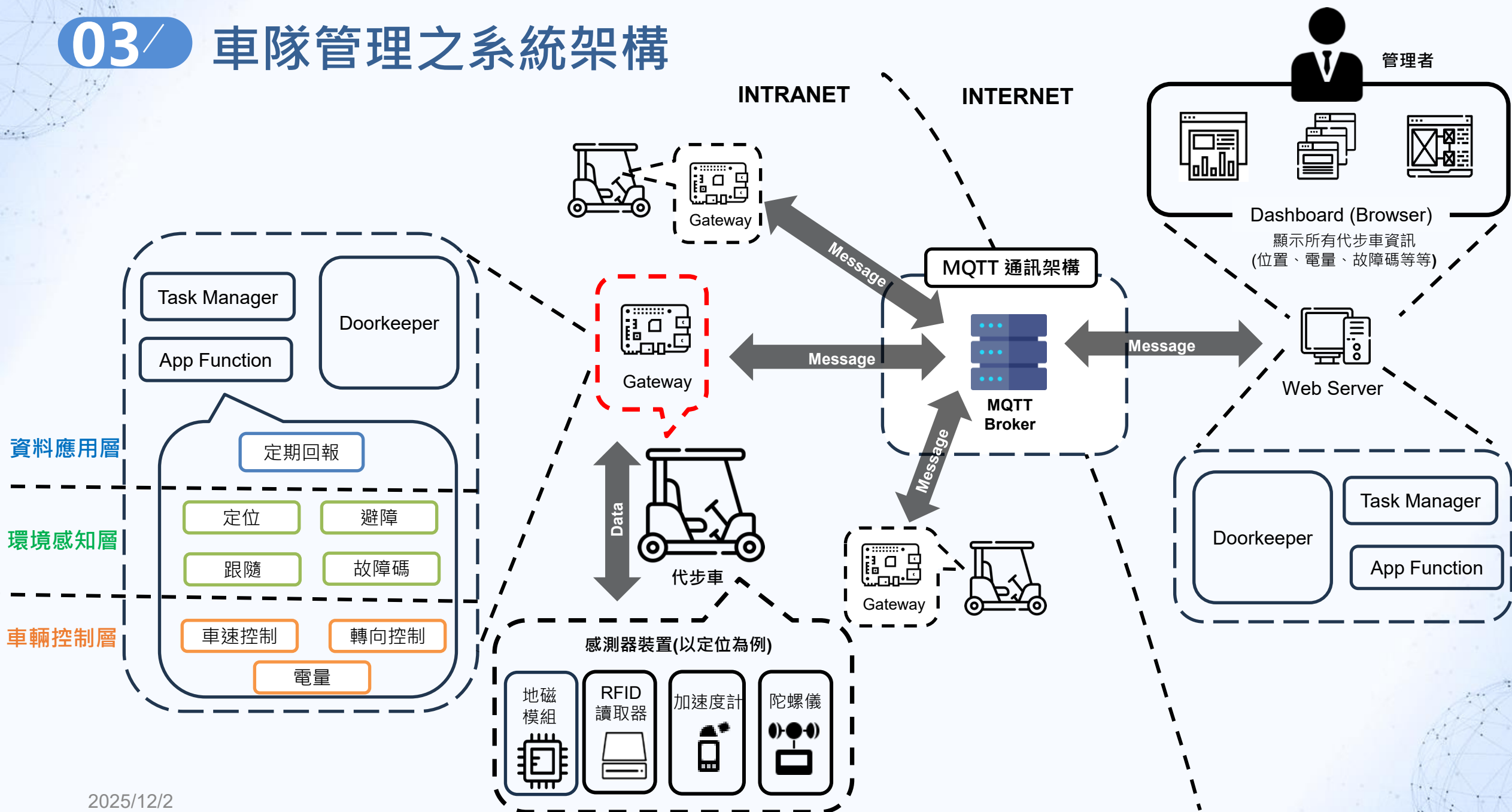
03/

電力管理之系統架構圖

- 場域中會有多台的閘道器，閘道器透過繼電器控制不同電器的狀態，並在閘道器中紀錄電器的狀態與用電等等資訊
- 使用者能從伺服器端控制電器狀態、進行客製化設定，並可在伺服器端的儀表板中顯示電器的用電資訊



03/ 車隊管理之系統架構



Outline

大綱>>

01/

物聯網閘道器概覽

02/

開放式閘道器介紹

03/

各項應用之系統架構圖

04/

開放式閘道器優化規劃

05/

參考資料與附錄

05/

開放式閘道器性能瓶頸分析

- 在收發訊息的過程中，系統會有多組操作同時更新同一張資料表（例如：mqtt_message_log、mqtt_task）。這些操作包含頻繁的 UPDATE、INSERT 指令，導致資料庫性能下降，主要原因如下：
 - 鎖競爭：對相同的資料表同時進行多個操作，導致資料表出現鎖競爭，增加了操作的執行時間
 - 頻繁更新：多筆資料同時對同一張表進行更新操作，導致系統的執行時間變長，出現性能瓶頸
- 解決方案：
 - 減少資料庫讀寫次數：使用**批次處理**，合併多筆 UPDATE、INSERT 操作等等，以減少鎖定等待時間；該方法可能導致**訊息處理的即時性降低**，可考慮設定批次處理時間間隔維持訊息處理的即時性
 - 減少資料庫連線次數：採用**資料庫連接池 (connection pooling)** 技術，通過維護多組可重用的資料庫連接，來減少應用程式與資料庫之間頻繁的連接和斷開操作。當應用程式需要與資料庫進行互動時，從連接池中取用預先建立好的連線，完成後，連線會被歸還到連接池中供後續使用 [4]；該方法需考量連接池的配置與持續監控，以建立完整的資源管理策略，防止連接數量不足、資源浪費等情況

TIMER_WAIT	LOCK_TIME	SQL_TEXT	CURRENT_SCHEMA
895700000	4000000	SHOW INDEX FROM `performance_schem...	NULL
216400000	2000000	SELECT * FROM performance_schema.ev...	NULL
420800000	4000000	SELECT * FROM `mqtt_message_log` WH...	fleet_gateway
362600000	2000000	UPDATE `mqtt_message_log` SET `mess...	fleet_gateway

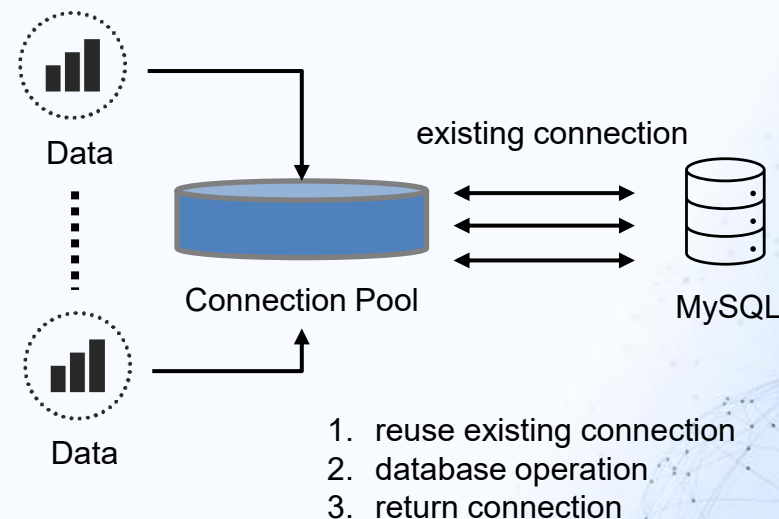
- 截圖中 mqtt_message_log 同時有查詢與更新的操作，可以看到 TIMER_WAIT 和 LOCK_TIME 欄位顯示操作的執行與等待時間 (奈秒)：
 - TIMER_WAIT：完成所有操作的總執行時間，包含查詢處理時間、I/O 操作時間，以及因等待其他資源而產生的延遲
 - LOCK_TIME：在執行之前因等待其他鎖定操作而造成的時間

05/ 導入 Connection Pool 技術

- 連續 10,000 次 Insert、Update、Delete、Select 資料表操作，比較導入資料庫連接池 (connection pool) 技術後資料庫連線、實際操作資料表與關閉資料庫連線時間
- 導入 connection pool 技術後：
 - 建立資料庫連線：平均耗時 0.002 毫秒，耗時減少 99.98%
 - Insert：0.981 ms、Update：0.922 ms、Delete：0.951 ms、Select：0.139 ms
 - 關閉/返回資料庫連線：平均耗時 0.056 毫秒，耗時減少 65.64%
- 經測試，導入connection pool 後可大幅降低資料庫連線開銷時間，也會影響資料指標 (data cursor) 所需時間

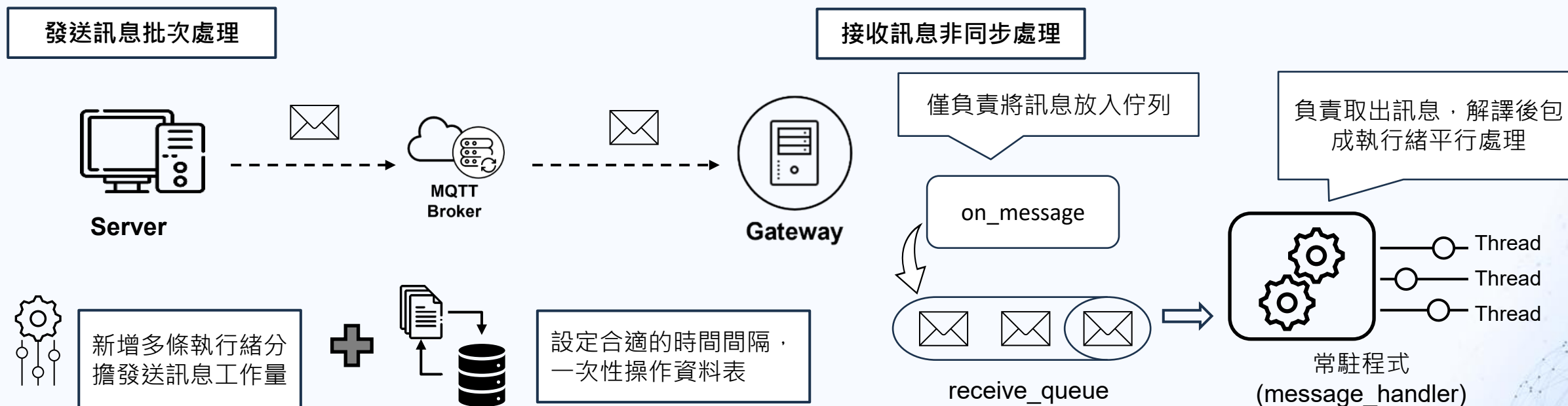
Method	Original	Connection Pool
建立連線	10.732 ms	0.002 ms
Insert	0.826 ms	0.812 ms
Update	0.601 ms	0.593 ms
Delete	0.497 ms	0.490 ms
Select	0.085 ms	0.091 ms
關閉連線	0.163 ms	0.056 ms

Data cursor	Original	Connection Pool
Insert	2.646 ms	0.096 ms
Update	0.168 ms	0.268 ms
Delete	0.144 ms	0.298 ms
Select	0.112 ms	0.090 ms



05/ 訊息收發優化

- 訊息接收的非同步處理 (Asynchronous Processing) :
 - 訊息遺失：提出的通訊架構中，利用套件中 'on_message' 的回調函式 (callback function)，處理接收到的訊息。原先設計在此函式上進行訊息解譯並將訊息包成執行緒做後續資料表操作，因此導致回調函式處理時間增加，造成部份訊息遺失
 - 解決方案：將收到的訊息存入佇列，並建立一個常駐程式來處理佇列中的訊息
- 訊息發送優化規劃：
 - 多執行緒：增加多個執行緒平行處理發送訊息任務
 - 批次處理：某些相同操作，如：發送訊息時的 INSERT 操作，可累積一定的資料量再批次進行



參考資料

[1] Q. Zhu, R. Wang, Q. Chen, Y. Liu, and W. Qin, "IOT Gateway: Bridging wireless sensor networks into internet of things," IEEE/IFIP International Conference on Embedded and Ubiquitous Computing, pages 347-352, Hong Kong, China, December 11-13, 2010.

[2] B. Kang, D. Kim, and H. Choo, "Internet of everything: A large-scale autonomic IoT gateway," IEEE Transactions on Multi-Scale Computing Systems, Volume 3, Issue 3, pages 206-214, May 18, 2017.

[3] https://yuanchieh.page/posts/2020/2020-01-14_redis-lock-redlock-%E5%8E%9F%E7%90%86%E5%88%86%E6%9E%90%E8%88%87%E5%AF%A6%E4%BD%9C/

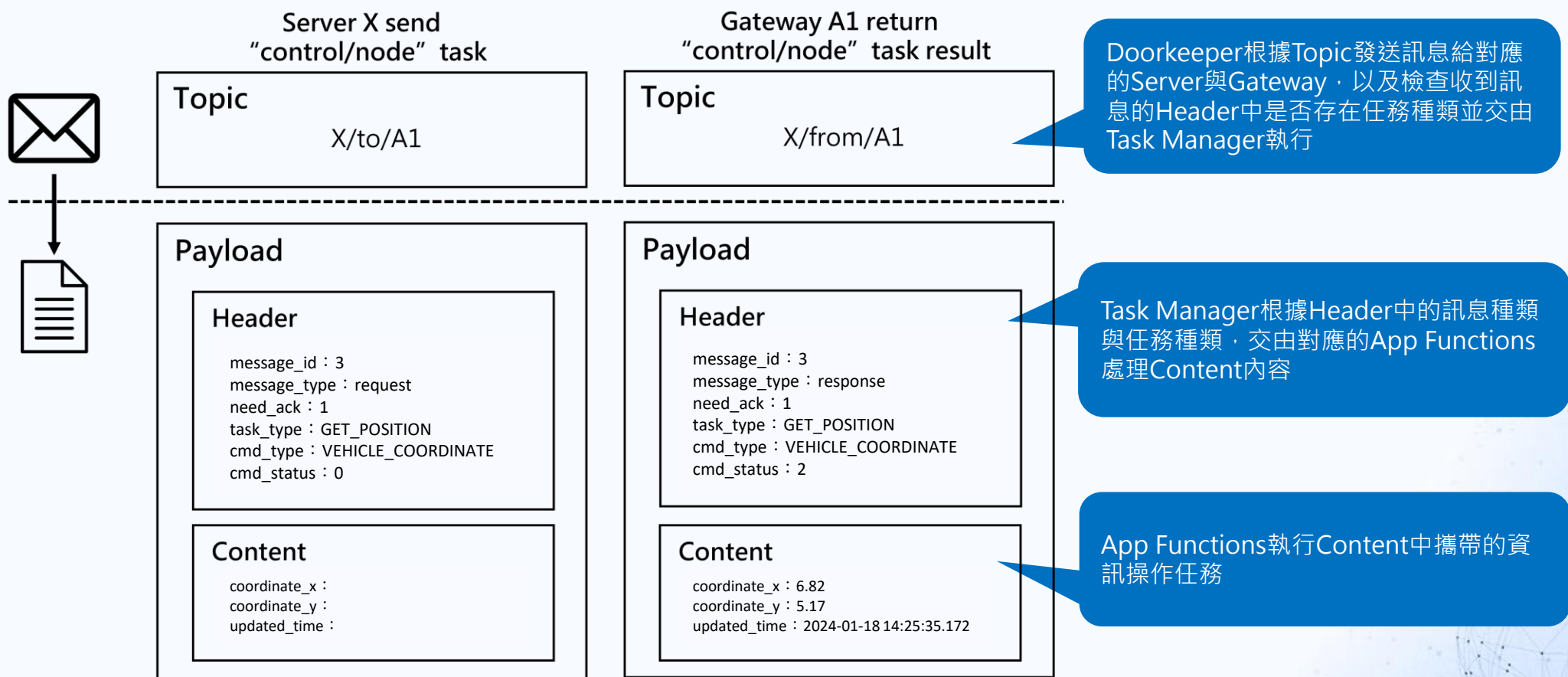
[4] <https://stackoverflow.blog/2020/10/14/improve-database-performance-with-connection-pooling/>

附錄一、開發環境

開發設備		規格	圖示
硬體	Gateway	RaspberryPi 4 Model B (8GB Ram)	
	Server、MQTT Broker	ASUS BM5270, Intel Core2 Q8400 2.7GHz, 10 GB RAM	
軟體	MQTT Protocol	MQTT protocol versions 5.0	
	Web 應用框架	Flask v 3.0.3	
	Web 伺服器	Apache v 2.4.0	
	資料庫	MySQL Community 8.0.3 CE	
	後端開發語言	Python 3.7	
	前端開發語言	HTML5、CSS3、Javascript (ES15)	

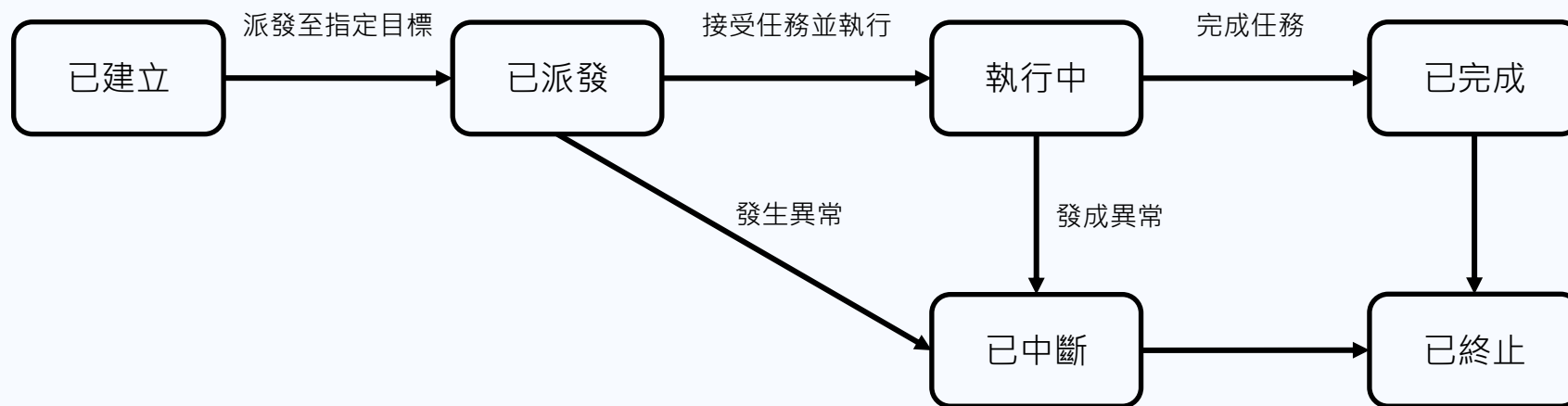
附錄二、訊息結構介紹

- 一則訊息會分為Topic與Payload，其中Payload中包含Header與Content，Header中記錄訊息、任務種類、指令種類等簡介資訊，而Content中則是任務執行的細節與狀態，而通用架構中不同的角色會根據需求使用部分訊息內容



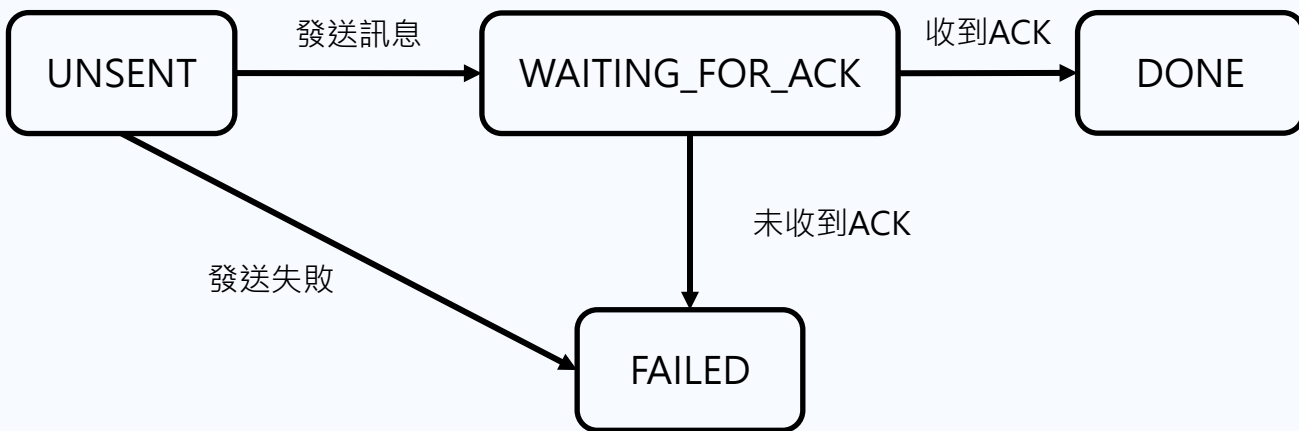
附錄三、任務狀態流程圖

- 任務狀態分為6種
 - 已建立：當任務建立後，已建立為任務初始狀態
 - 已派發：當任務派發至指定目標時 (Server或Gateway) ，Task Manager更改任務狀態為已派發
 - 執行中：接受任務並確認可執行後，Doorkeeper更改任務狀態為執行中
 - 已完成：完成任務時，Task Manager更改任務狀態為已完成
 - 已中斷：當任務發生異常狀況時，Task Manager切換任務狀態為已中斷
 - 已終止：當Server與Gateway的任務狀態一致時，Doorkeeper切換任務狀態為已終止，即為任務的結束狀態
- 以正常的即時點位控制為例：Server建立即時控制點位任務(已建立) 並派發給Gateway (已派發)，Gateway開始更新點位狀態 (已執行)，當狀態更新完成 (已完成) 後Gateway將結果回傳並確認Server收到結果 (已終止)



附錄四、訊息狀態流程圖

- 待發送與等待ACK的訊息皆存在“mqtt_message”資料表
- 訊息狀態分為 3 種，由“message_status”欄位紀錄：
 - UNSENT：尚未取出發送的訊息
 - WAITING_FOR_ACK：等待回應ACK的訊息
 - DONE：接收到ACK的訊息
- Timeout (5秒)：成功發送訊息後會將訊息狀態轉成 WAITING_FOR_ACK，此狀態下的訊息 5 秒內要收到回應的 ACK 確保已正確傳遞，若未收到回應的 ACK，會再次發送該訊息

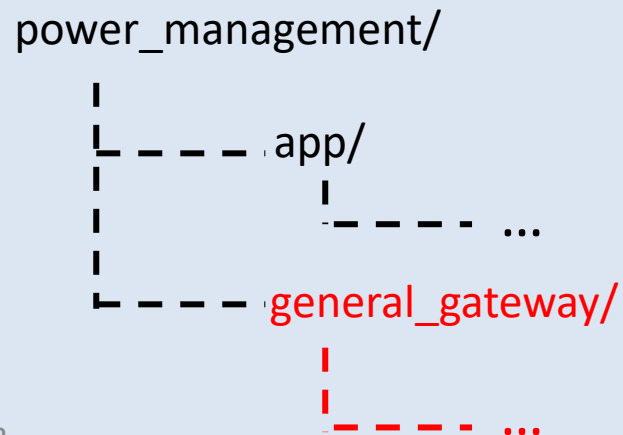


- 訊息重新發送機制：
 1. 距離首次發送時間未超過 5 分鐘
 2. 重新發送次數未超過 5 次

附錄五、如何將開放式閘道器導入現有專案 – 以電力管理為例

- 目的：建立開放式閘道器模組，只要導入此模組至專案中便可使用
- 在電力管理專案中導入通用型閘道器模組，完成設定即可啟動
 - a. 設定資料庫連線
 - b. 設定Broker位置、Server與Gateway ID與角色 (Server/Gateway)
 - c. 啟動通用閘道器模組

電力管理專案架構



電力管理

power_management/app/api/project.py

```
# 呼叫通用閘道器模組
general_gateway = GeneralGateway().set_db_connection(
    DB_CONFIG['host'],
    DB_CONFIG['user'],
    DB_CONFIG['password'],
    DB_CONFIG['db']
).set_basic_config(
    BROKER_HOST,
    SERVER_ID,
    GATEWAY_ID,
    ROLE
)

general_gateway.start()
```

附錄六、建立任務並打包為訊息 – 以點位控制為例

a.

```
content = json.dumps(  
    {  
        'gateway_sn': GATEWAY_SN,  
        'node_sn': node_sn,  
        'node_status': node_status  
    }  
)  
  
mqtt_task = MQTTTask(  
    gateway_sn=GATEWAY_SN,  
    task_type=TaskType.SWITCH_NODE,  
    task_status=TaskStatus.CREATED,  
    content=content  
)  
mqtt_task.task_sn = insert_mqtt_task(mqtt_task)
```

b.

```
if ROLE == "Gateway":  
    direct = MessageDirection.FROM  
else:  
    direct = MessageDirection.TO  
  
result = add_to_mqtt_message(  
    need_ack=True,  
    message_type=MessageType.REQUEST,  
    mqtt_task=mqtt_task,  
    direction=direct,  
    gateway_id=GATEWAY_ID  
)
```

- 先建立任務，再打包為訊息加入訊息資料表 (mqtt_message)
 - a. 建立任務，其中會包含任務的種類 (task_type)、執行狀態 (task_status)、以及執行內容 (content)，並將任務加入 mqtt_task 資料表中
 - b. 將任務以及訊息傳輸時需要的資訊，透過 add_to_mqtt_message() 儲存到 mqtt_message 資料表，其中包含是否需要接收方回傳ACK (need_ack)、訊息種類 (message_type)、任務 (mqtt_message)、傳送方向 (direction)、傳送的gateway (gateway_id)

附錄七、Doorkeeper程式通則 – 發送訊息

```
# 發送訊息的人，排程執行（不斷撈取訊息）
def _send_message(self):
    print()
    try:
        a. messages = self.fetch_need_publish_messages()

        if messages is not None:
            b. for message in messages:
                message_id = message.get('message_id')
                topic = message.get('topic')
                payload = message.get('payload')

                self.client.publish(topic, payload, qos=self.mqtt_qos)

                c. update_mqtt_message_status_by_message_id(
                    message_id,
                    status=MessageStatus.DONE if bool(message.get(
                        'is_need_ack')) is False else MessageStatus.WAITING_FOR_ACK
                )
    except BaseException as e:
        trace_error(e)
        return False
```

- 此函式負責找出mqtt_message中需發送的訊息，並依序發送
 - a. 透過fetch_need_publish_message()取得資料庫中所有需發送的訊息
 - b. 依序取得訊息中的topic和payload並透過MQTT發送
 - c. 更新訊息的狀態
 - 若訊息需要接收方回覆ACK，則狀態從UNSENT->WAITING_FOR_ACK
 - 若訊息不需要接收方回覆ACK，則狀態從UNSENT->DONE

附錄八、Doorkeeper程式通則 – 接收訊息

```
def deal_message(self, mqtt_message):
    try:
        a. if mqtt_message.message_type == MessageType.REQUEST \
            or mqtt_message.message_type == MessageType.RESPONSE:
            if mqtt_message.is_need_ack:
                self.create_ack_message(mqtt_message)

            b. mqtt_task = read_task_payload(
                mqtt_message.topic,
                mqtt_message.payload
            )

            if mqtt_message.message_type == MessageType.REQUEST:
                insert_mqtt_task(mqtt_task)
            else:
                update_mqtt_task_by_task_sn_and_gateway_sn(mqtt_task)

            c. TaskManager().add_task_sn_to_task_list(
                mqtt_task.task_sn, mqtt_message.message_type
            )

            d. elif mqtt_message.message_type == MessageType.ACK:
                self.deal_ack_message(mqtt_message)
            else:
                raise Exception("Unknown message type")
```

- 當透過MQTT接收到訊息時，會呼叫此函式處理訊息
 - a. 判斷接收到的訊息種類 (REQUEST, RESPONSE, ACK)，並做對應的後續處理
 - b. 讀取訊息內容的任務，將任務新增或更新資料庫
 - c. 將要處理的任務交由Task Manager進行後續任務處理
 - d. 處理ACK訊息，透過deal_ack_message()將訊息狀態從WAITING_FOR_ACK->DONE

附錄九、Task Manager程式通則

```
def _deal_task(self):
    try:
        a. tasks = self.fetch_need_deal_tasks()

        if tasks:
            b. for mqtt_task in tasks:

                task_timeout = data_plus_seconds(
                    mqtt_task.created_time, mqtt_task.task_timeout)
                now = datetime.now()

                if now > task_timeout:
                    mqtt_task.task_status = TaskStatus.FAILED
                    update_mqtt_task_status_by_mqtt_task(mqtt_task=mqtt_task)
                else:
                    message_type = MessageType(self._todo_task_dict.get(mqtt_task.task_sn))
                    mqtt_task = read_task_content(mqtt_task=mqtt_task)

                    self._app_executor.app_dispatch(mqtt_task, message_type)

                TaskManager().remove_task_sn_from_task_list(mqtt_task.task_sn)
    except BaseException as e:
        trace_error(e)
    return False
```

- Task Manager根據Doorkeeper傳入的任務依序管理任務進度與交由App functions處理
 - a. 透過fetch_need_deal_tasks()取得任務資料表 (mqtt_task) 中所有需處理的任務
 - b. 判斷任務是否過期，若未過期則會交由App function處理

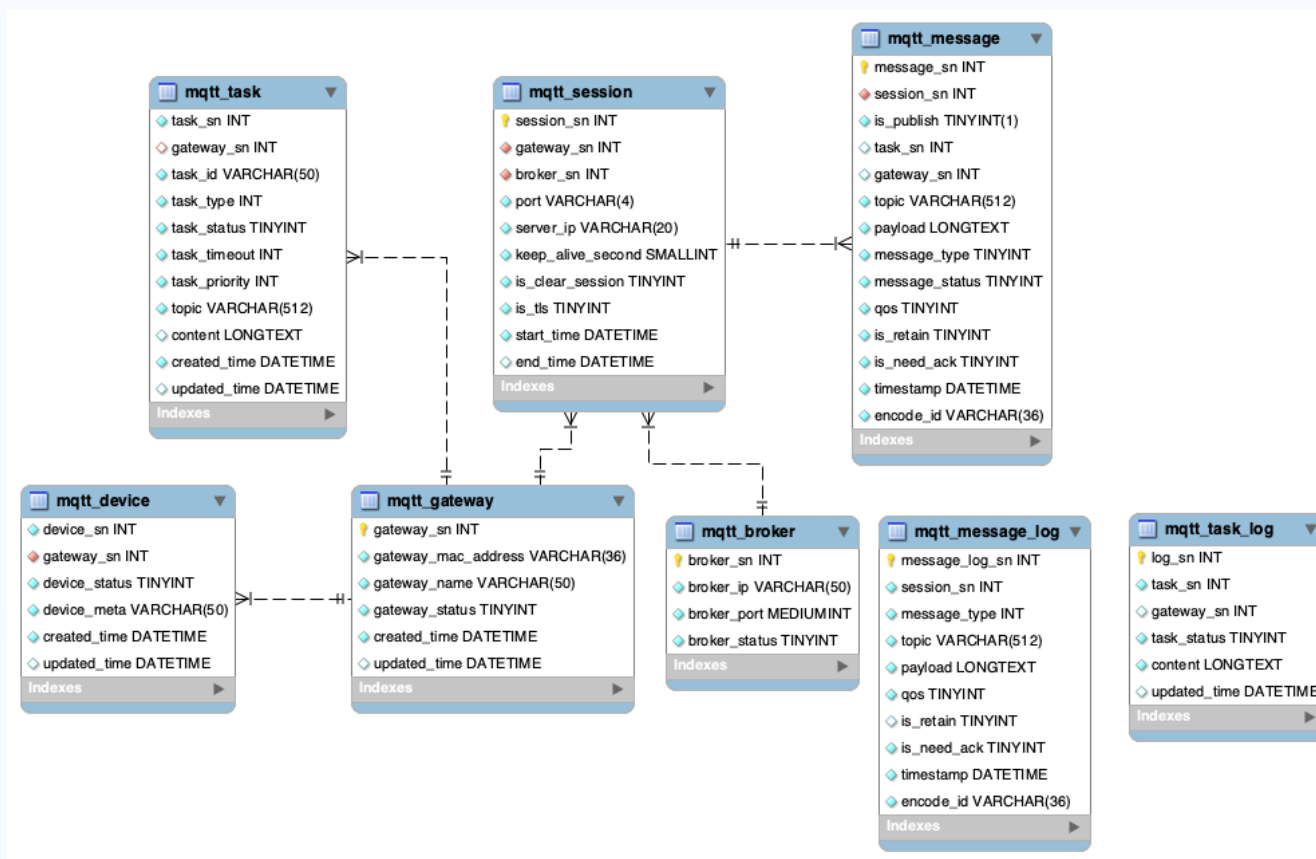
附錄十、App function業務邏輯設計— 以點位控制為例

- 以即時點位控制 (SWITCH_NODE) 的業務邏輯為例
 - a. 伺服器傳送點位控制訊息給閘道器 (REQUEST)，當閘道器接收會執行 `app_switch_node()` 函式控制點位狀態，並回傳控制結果給伺服器
 - b. 伺服器接收到閘道器回傳的結果 (RESPONSE)，會執行 `app_switch_node_result()` 檢查結果，即完成一次的即時點位控制

```
def app_dispatch(self, mqtt_task: MQTTTask, message_type: MessageType):  
    try:  
        a. if mqtt_task.task_type is TaskType.SWITCH_NODE and message_type is MessageType.REQUEST:  
            self.app_switch_node(mqtt_task)  
        b. elif mqtt_task.task_type is TaskType.SWITCH_NODE and message_type is MessageType.RESPONSE:  
            self.app_switch_node_result(mqtt_task)
```

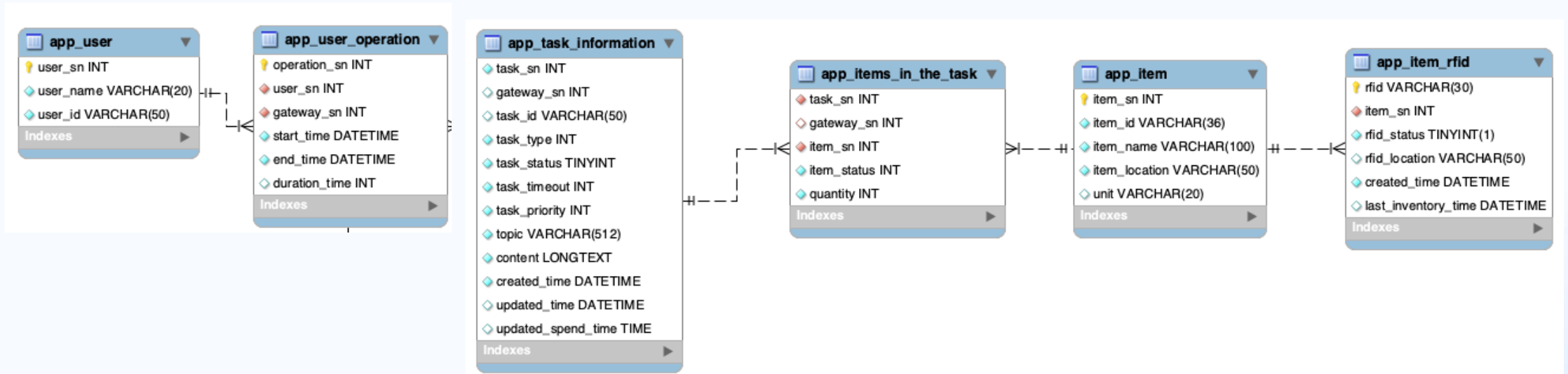
附錄十一、通用的資料表

- **mqtt_資料表**：開放式閘道器通訊模組中通用的資料表，用於管理訊息與任務，例如保存收到的訊息
- 相關操作函式擺放於common/db_util.py中



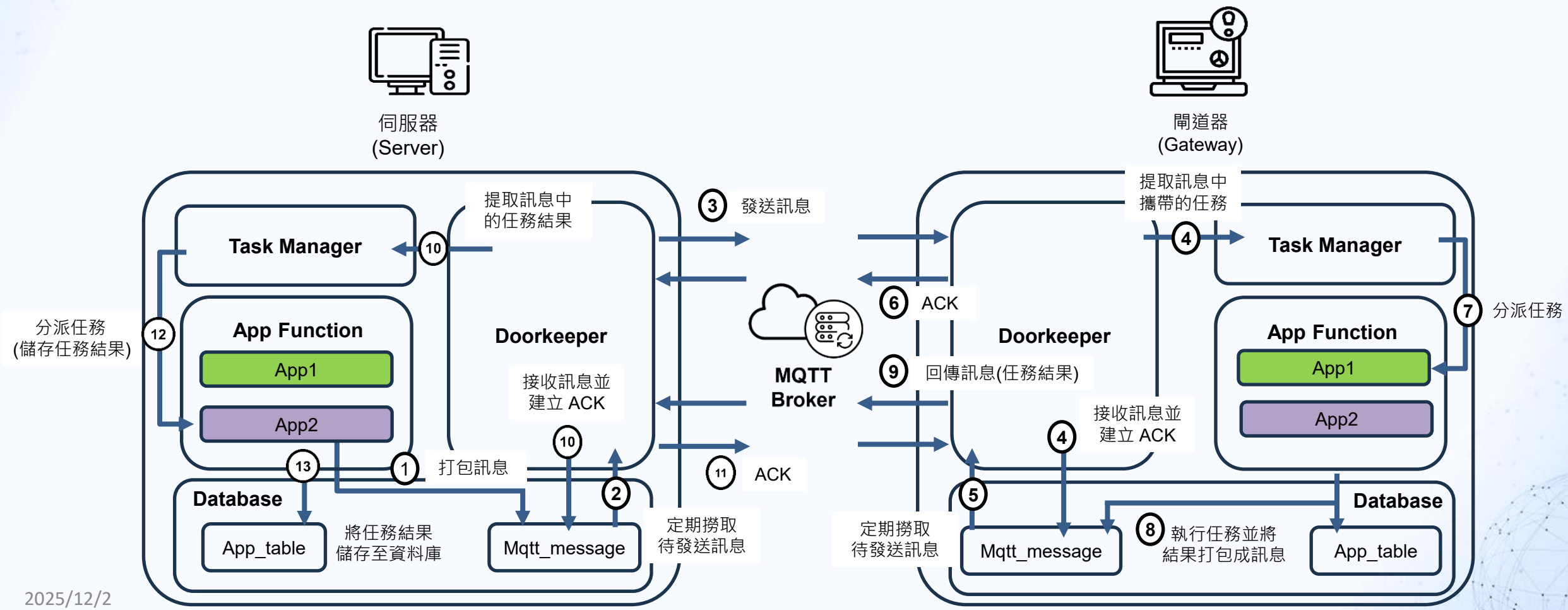
附錄十二、特定情境的資料表-以倉儲為例

- **app_資料表**：針對特定領域用的資料表，例如紀錄盤點到的RFID Tag
- 相關操作函式擺放於services下，並以情境命名 (db_warehouse.py)



附錄十三、雙向訊息傳輸流程

- 伺服器與閘道器採用相同的架構進行訊息傳輸與裝置控管，雙向訊息傳輸流程如架構圖所示



附錄十四、紀錄訊息發送耗時

發送訊息有 2 種模式：

- ① 訊息發送端：於記憶體中建立訊息，每發送一則便新增至 'mqtt_message_log'
- ② 接收訊息後回傳：建立 Ack 存放在資料庫，由排程定期取出回傳給發送端

①

- 1) 建立訊息內容、紀錄訊息狀態，包含發送時間、任務內容等資訊，耗時 ≈ 0 ，可忽略不計
- 2) 發送訊息，平均耗時 0.001 秒
- 3) 插入 message log 資料表，平均耗時 0.033 秒

發送過程 3 個階段平均耗時

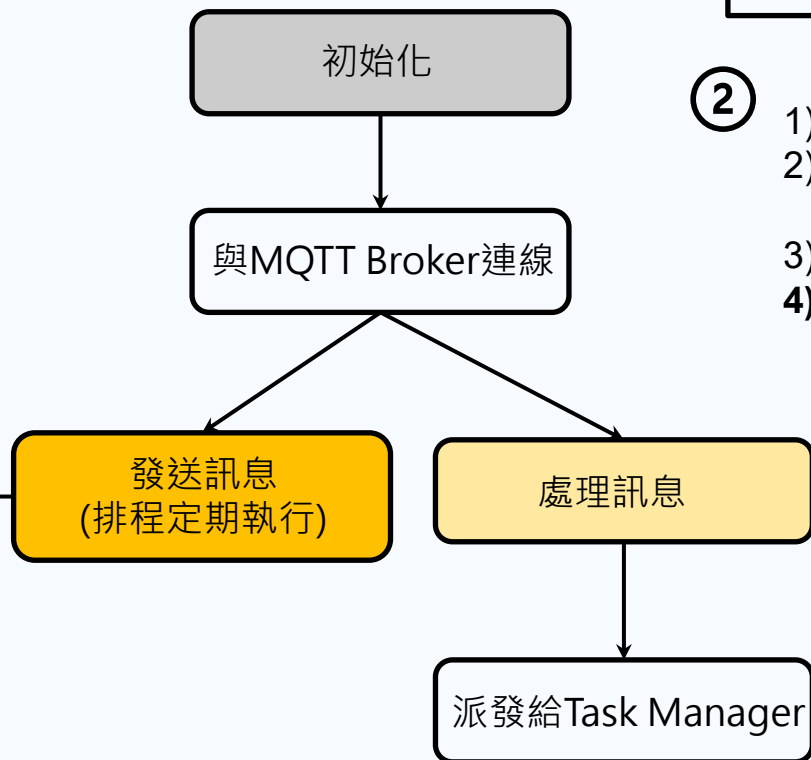
1	2	3
0.000	0.001	0.033



Insert 到 'mqtt_message_log' 耗時：

Min	Median	Max
0.000	0.031	0.343

Doorkeeper物件



量測 2 種發送訊息模式之耗時：

- ① 發送每則訊息皆會經過 1、2、3 階段，因此理論上每秒可以發送 29.4 則
 $1 / (0.001 + 0.033) = 29.4$
- ② 發送訊息會 query 一次待發送訊息，並逐一發送，因此理論上每秒可以發送 44.0 則
 $(1 - 0.075) / (0.001 + 0.020) = 44.0$

②

- 1) 擷取所有待發送訊息，平均耗時 0.075 秒
- 2) 紀錄訊息狀態，包含發送時間、任務內容等資訊，耗時 ≈ 0 ，可忽略不計
- 3) 發送訊息，平均耗時 0.001 秒
- 4) 更新 message log 資料表，平均耗時 0.020 秒

發送過程 4 個階段平均耗時

1	2	3	4
0.075	0.000	0.001	0.020

Update 'mqtt_message_log' 耗時：

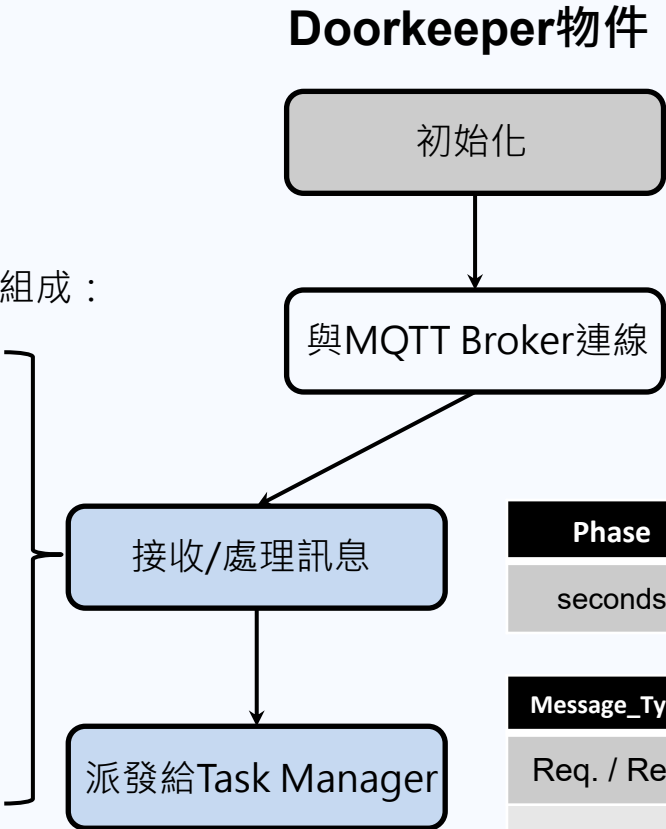
Min	Median	Max
0.000	0.015	0.218

附錄十五、紀錄接收訊息耗時

- 根據量測結果，平均每秒約可以處理 2 則訊息，但因為同時最多有 15 個執行緒在處理訊息，因此：
 - ① Request / Response：每秒可以處理 30.6 則
 - ② Ack：每秒可以處理 37.1 則

負責處理訊息的程式 “def deal_message(self)” 由 2 個步驟組成：

- 1) 檢查訊息 topic 與 payload 格式，提取訊息內容，耗時可忽略不計
- 2) 處理訊息：
 - ① Request/Response：建立 Ack 訊息 (新增資料到 mqtt_message、message_log)、讀取任務內容、新增/更新任務資料表，平均耗時 0.490 秒
 - ② Ack：更新 mqtt task、message log 資料表，平均耗時 0.404 秒



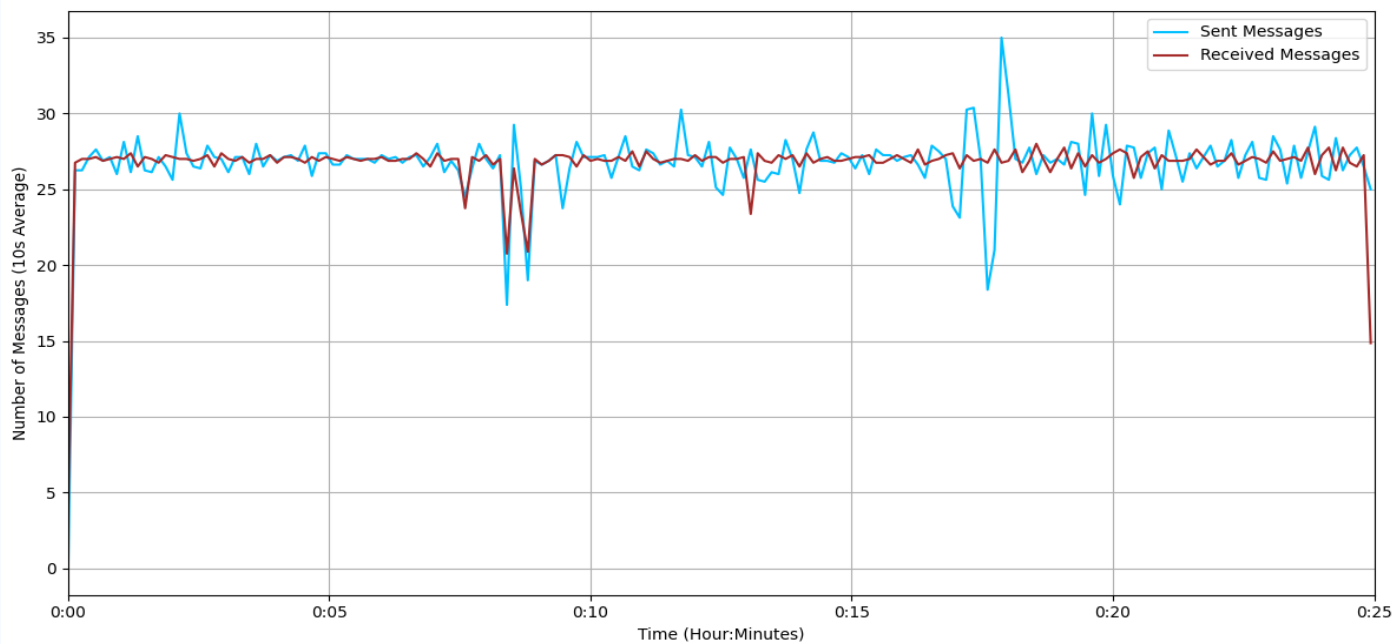
在訊息發送與接收的過程中，資料庫的 INSERT 和 UPDATE 操作為主要的效能瓶頸，因此改善資料庫操作耗時，可提升開放式閘道器模組的即時性與吞吐量

Phase	1	2-1	2-2
seconds	0.000	0.490	0.404

Message_Type	Min	Median	Max
Req. / Res.	0.015	0.453	2.015
Ack	0.217	0.395	0.662

附錄十六、閘道器端通訊吞吐量測試

- 測試方法：
 - 通訊模組運行後，初始化建立 100 則訊息存放在發送訊息的佇列不再生成，並固定發送此佇列訊息
 - 數據紀錄：發送與接收的訊息資訊皆記錄在 'mqtt_message_log' 資料表，透過 payload 欄位中時間 (發送時間) 與更新資料的時間 (接收時間) 計算測試期間模組的吞吐量
- 測試結果：
 - 測試時長 30 分鐘，期間發送 48,147 則訊息，接收 48,128 則訊息
 - 模組平均發送量 26.7 則/秒，接收量 26.7 則/秒

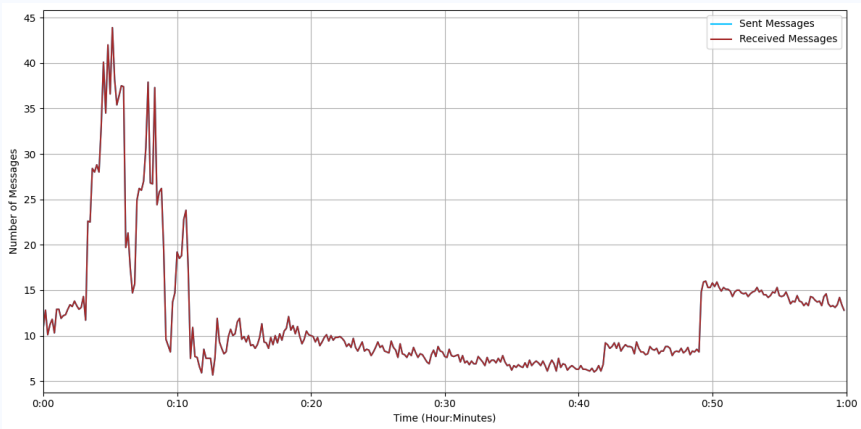


Method	Average	Max	Min	Median
發送	26.7484	37	1	27
接收	26.7377	49	1	27

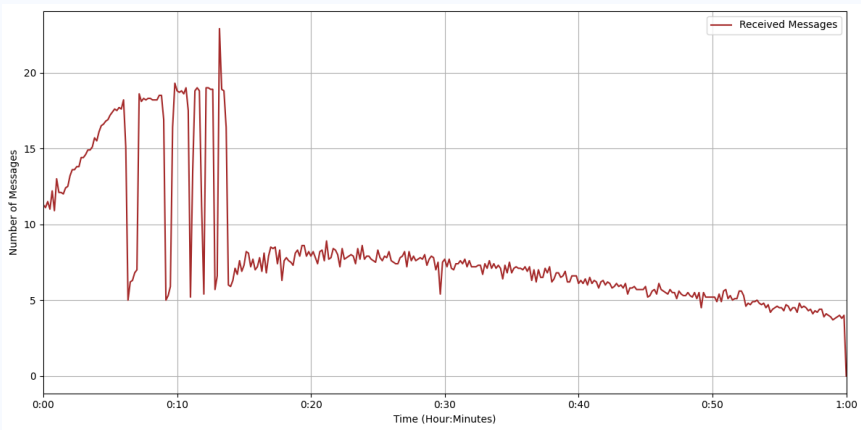
單位：則/秒

附錄十七、閘道器端吞吐量

- 測試開放式閘道器延展性實驗 1 中，3 個不同閘道器端的吞吐量實驗數據：

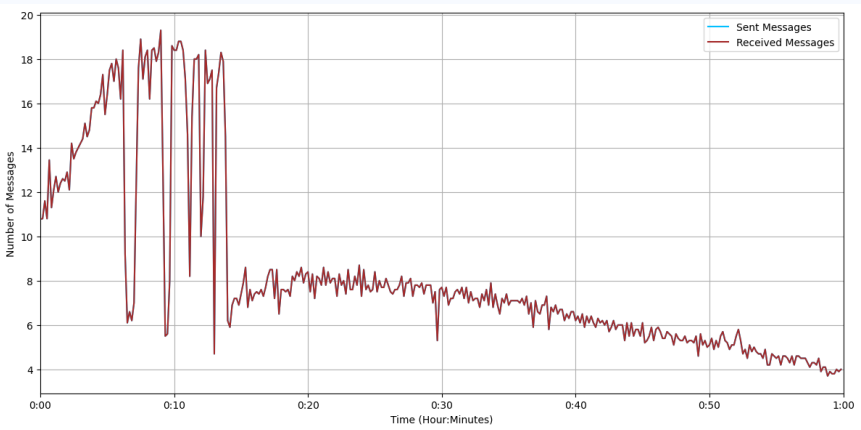


1 號閘道器



2 號閘道器

Gateway	Method	Average	Max	Min	Median
1	發送	9.9947	57	2	10
	接收	9.9947	57	2	10
2	發送	9.9965	25	2	10
	接收	9.9965	25	2	10
3	發送	9.9930	46	2	10
	接收	9.9930	49	2	10



3 號閘道器

單位：則/秒